Resilient Communication Networks

Designing Resilient Communication Networks

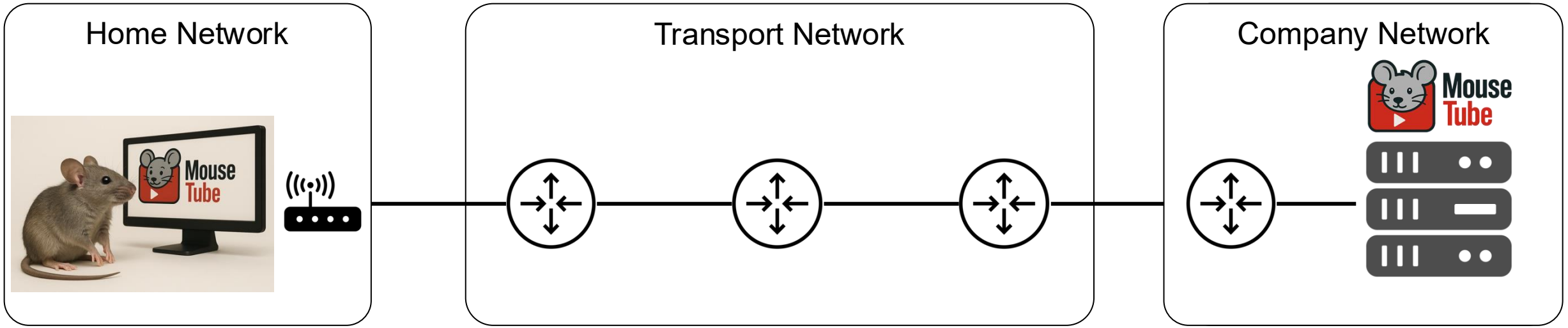
Redundancy Mechanisms – Peer-to-Peer Networks



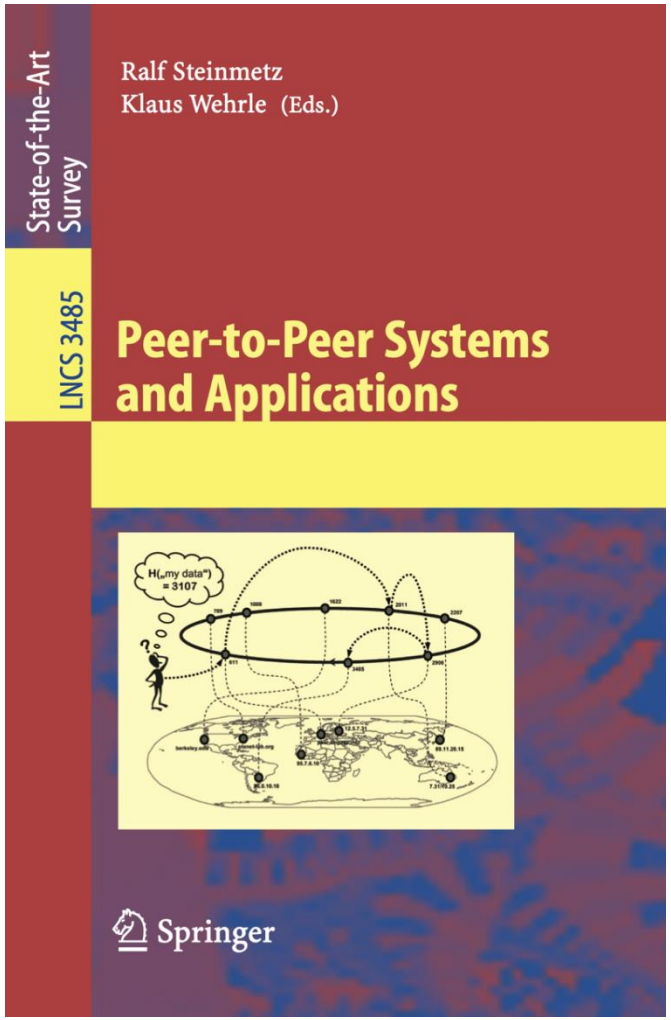
KOM – Communication Networks Lab
Technical University of Darmstadt
Prof. Dr. Björn Scheuermann (Director)
Dept. of Electrical Engineering and Information Technology
Dept. of Computer Science (adjunct Professor)
www.KOM.tu-darmstadt.de

Disclaimer: some of these slides have been derived from Günter Schäfer's lecture „Protection of Communication Infrastructures", Mathias Fischers Lecture „Resilient Networks", Matthias Hollick's Lecture „ Ringvorlesung KRITIS", and Ralf Steinmetz's Lecture "Communication Networks II"

What if we do not want to communicate to a server?



Peer-to-Peer: 9 Features



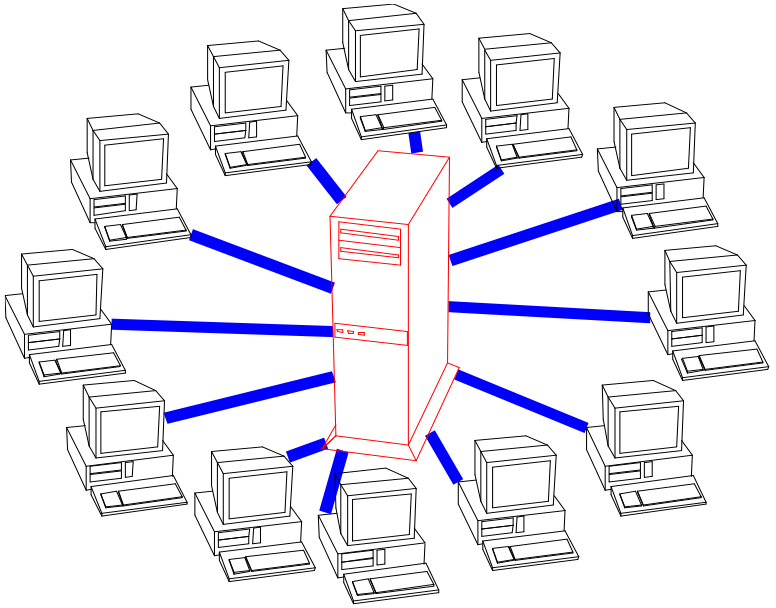
- 1. Relevant resources located at nodes (“peers”) at the edges of a network
- 2. Peers share their resources
- 3. Resource locations
  - Widely distributed
  - Most often largely replicated
- 4. Variable connectivity is the norm
- 5. Combined Client and Server functionality
- 6. Direct interaction (provision of services, e.g. file transfer) between peers (= “peer-to-peer”)
- 7. Peers have significant autonomy and mostly similar rights
- 8. No central control or centralized usage/provisioning of a service
- 9. Self-organizing system

Client / Server Model vs. P2P Technology

TECHNISCHE UNIVERSITÄT DARMSTADT

Main features of a Client / Server model:

- In principle, 1 server and N clients
- Clear distinction between server and client functionalities
- Simple search mechanism



Issue:

- Which server has the required information or file?

Solution:

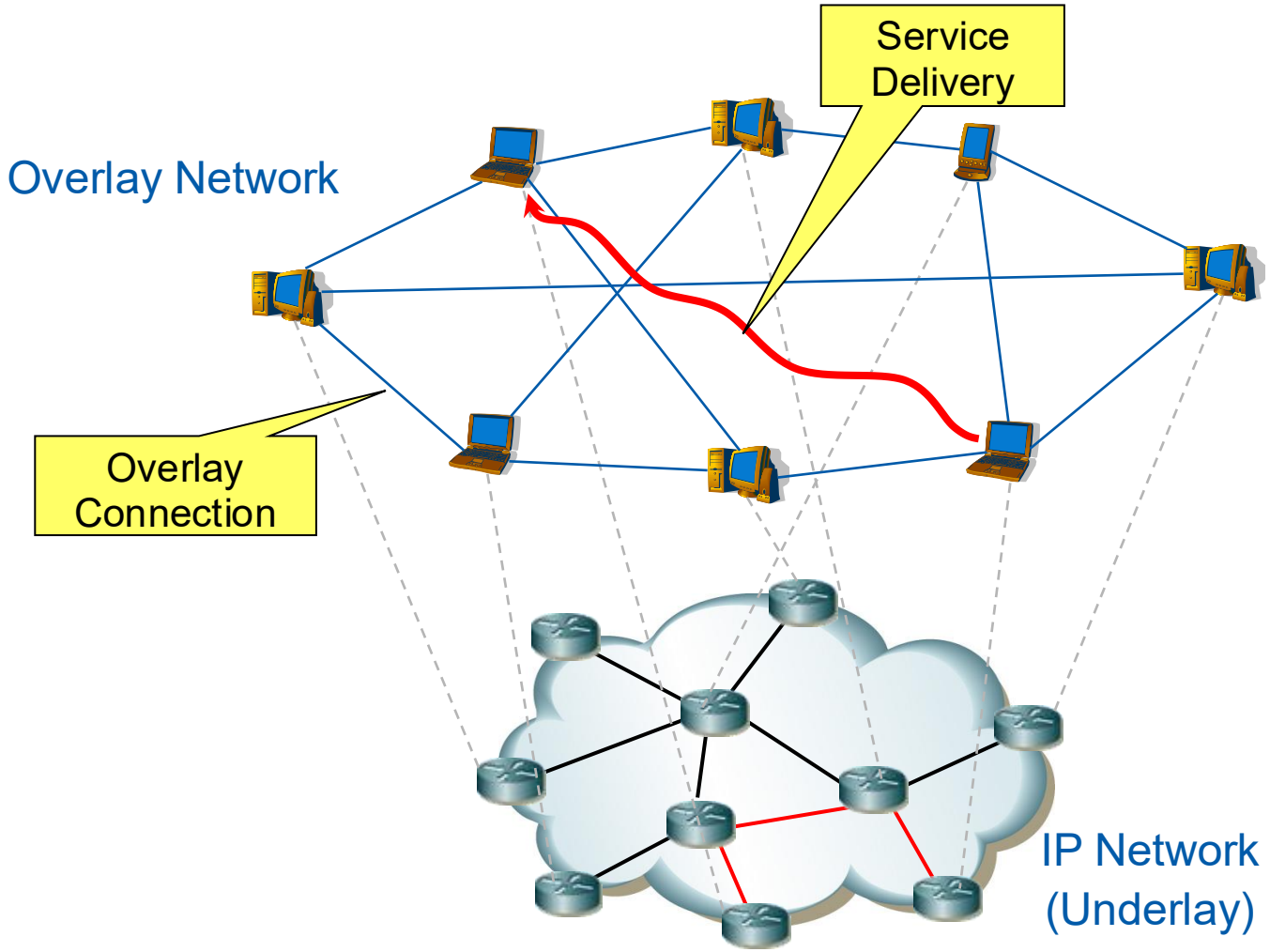
- Look it up on one server after another
  (or on Google, which does this for you)

Advantages:
Reliable, well known behavior

Drawbacks:
Server needs to provide (almost) all resources

What if we do not want to rely on client/server?

Fundamentals of Overlay Networks



Peers provide:

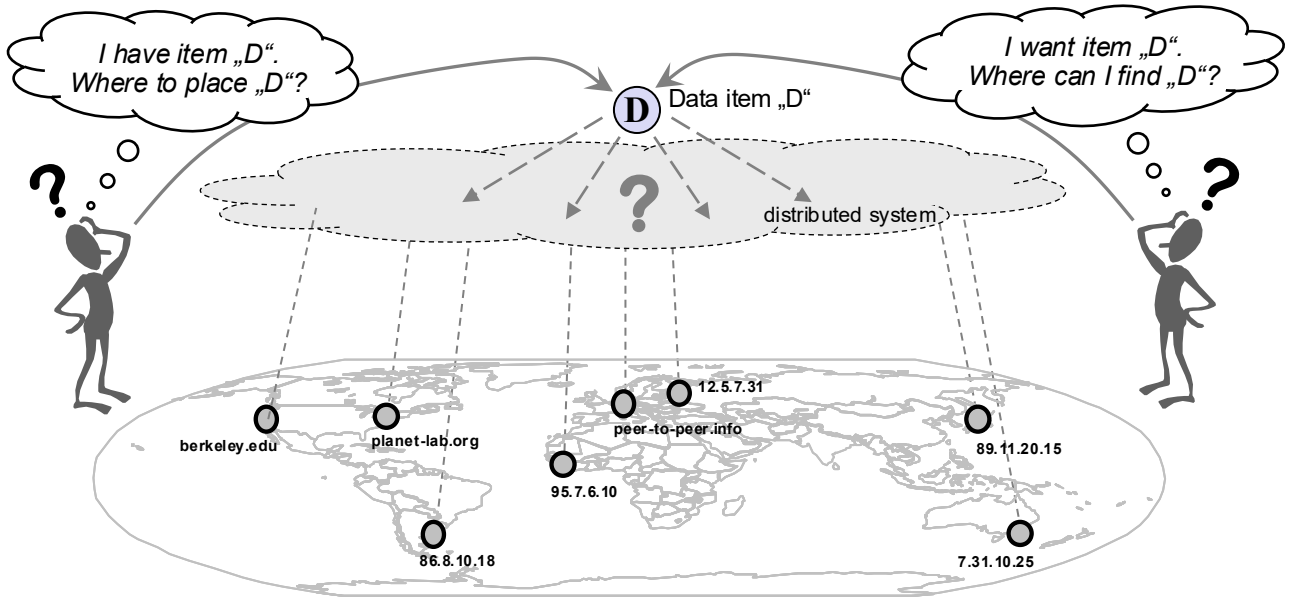
- Communications
- Processing
- Storage & Retrieval
- Interaction

Challenges in P2P related Data Management and Retrieval

TECHNISCHE UNIVERSITÄT DARMSTADT

Essential challenges in (most) Peer-to-Peer systems

- Location of a data item at distributed systems
  - Where should the item be stored by the provider?
  - How does a requester find the actual location of an item?
- Scalability
  - To keep the complexity for communication and storage “scalable”
- Robustness and resilience
  - In case of faults and frequent changes

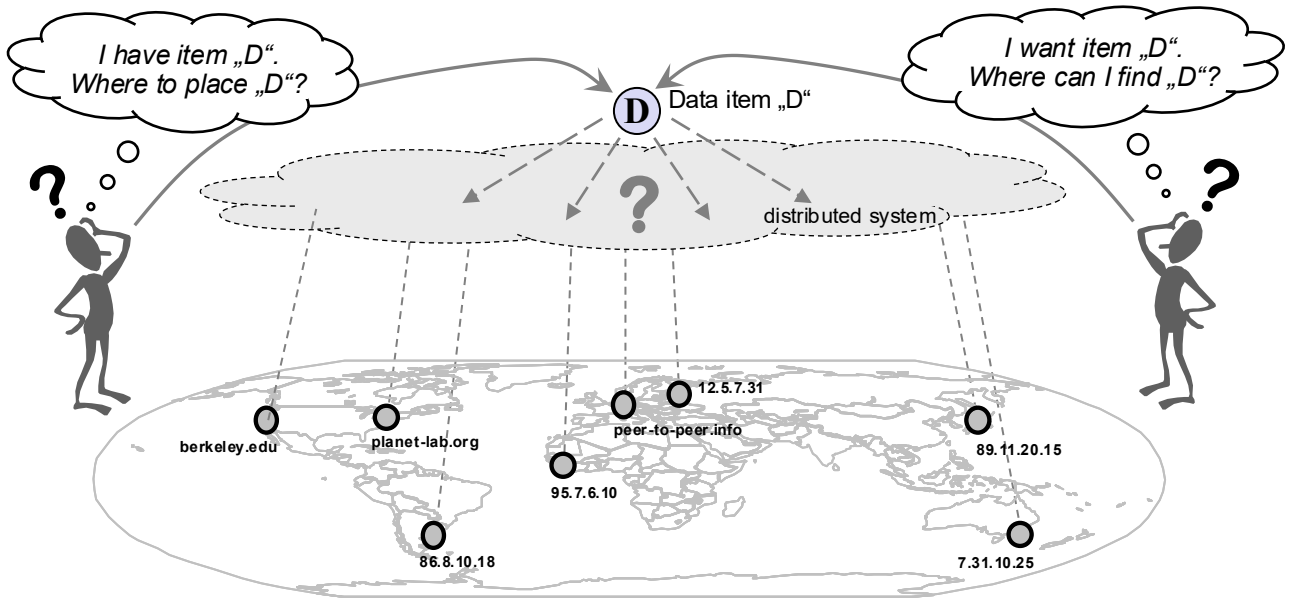


Strategies for Data Retrieval

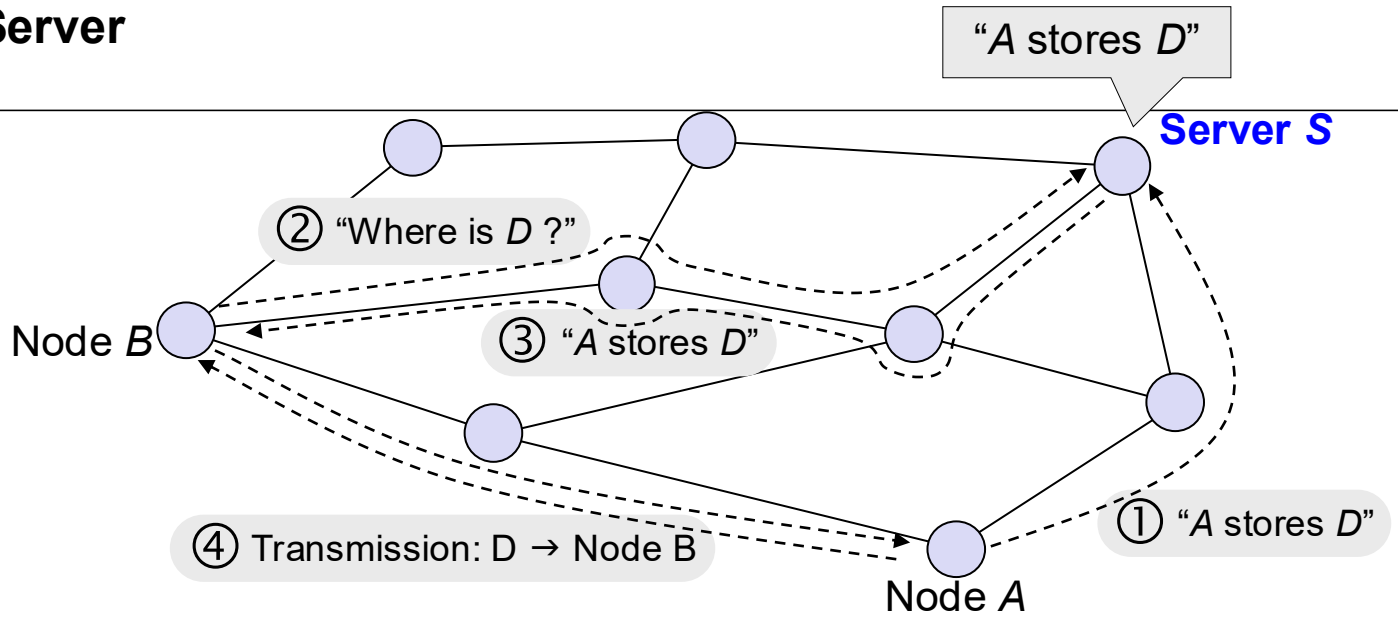
TECHNISCHE UNIVERSITÄT DARMSTADT

Strategies to store and retrieve data items in distributed systems

- Central server (**central indexing**)
- Flooding search (**local indexing**)
- Routing (**distributed indexing**)

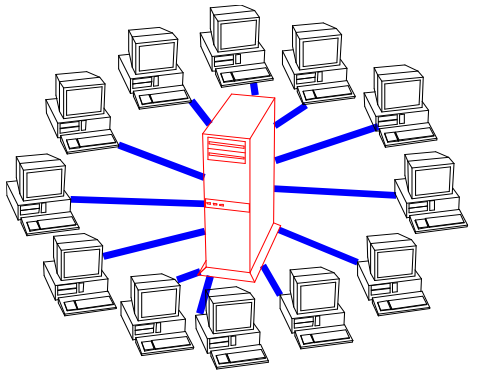


Strategies for Data Retrieval
Approach I: Central Server



Simple strategy :

- Central server stores information about locations (central index)
  - ① Node A (provider) tells server that it stores item D
  - ② Node B (requester) asks server S for location of D
  - ③ Server S tells B that node A stores item D
  - ④ Node B requests item D from node A



Approach I: Central Server

TECHNISCHE UNIVERSITÄT DARMSTADT

Advantages

- Search complexity of O(1) – “just ask the server”
- Complex and fuzzy queries are possible
- Simple and fast

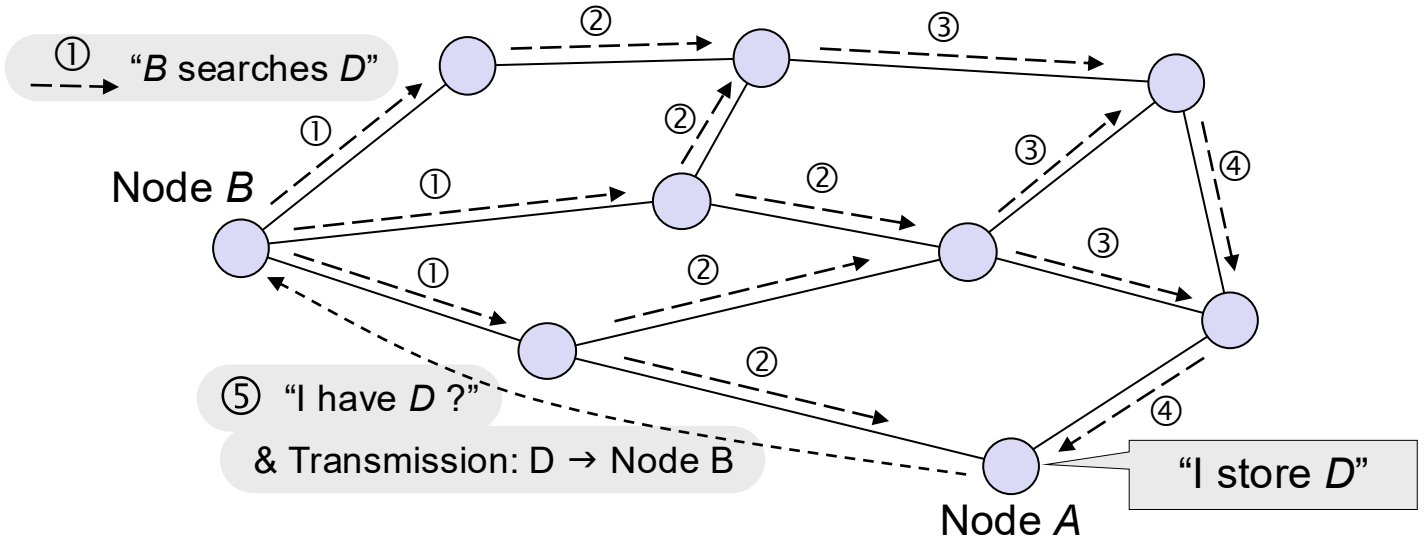
Challenges

- No intrinsic scalability
- Single point of failure or attack (at least without load balancing)

But overall, …

- Best principle for small and simple applications
- Or: massive distributed system acting as “central server” (e.g., Google)

Approach II: Flooding Search



Local Indexing Approach

- No information about location of data at intermediate systems
- Necessity for broad search
  - ① Node B (requester) asks neighboring nodes for item D
  - ②-④ Nodes forward request to further nodes (breadth-first search / flooding)
  - ⑤ Node A (provider of item D) sends D to requesting node B

Strategies for Data Retrieval Approach II: Flooding Search

TECHNISCHE UNIVERSITÄT DARMSTADT

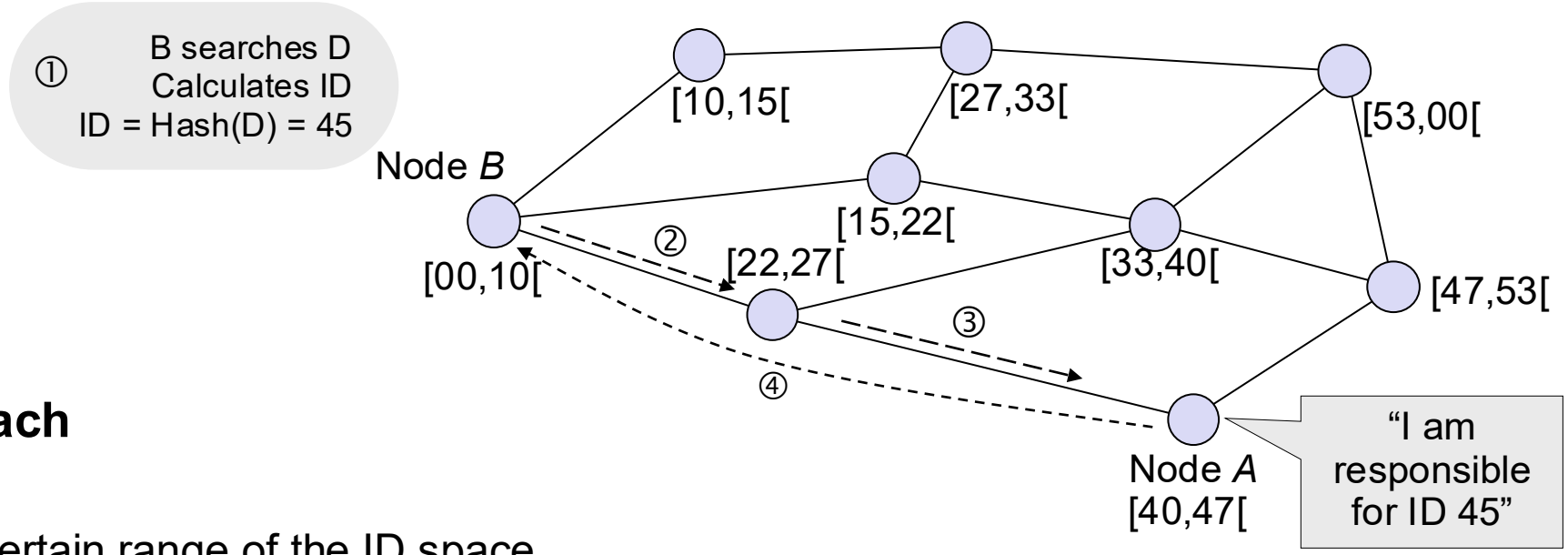
Fully Distributed Approach

- No global information available about location of any item
- Content only stored at respective node providing it

Retrieval of data

- No routing information for content
- Necessity to ask as many systems as possible / necessary
- Approach
  - Flooding: high traffic load on network, does not scale
  - Highest effort for searching
    - Quick search through large areas
    - Many messages needed for unique identification

Strategies for Data Retrieval Approach III: Routing (Distributed Indexing)



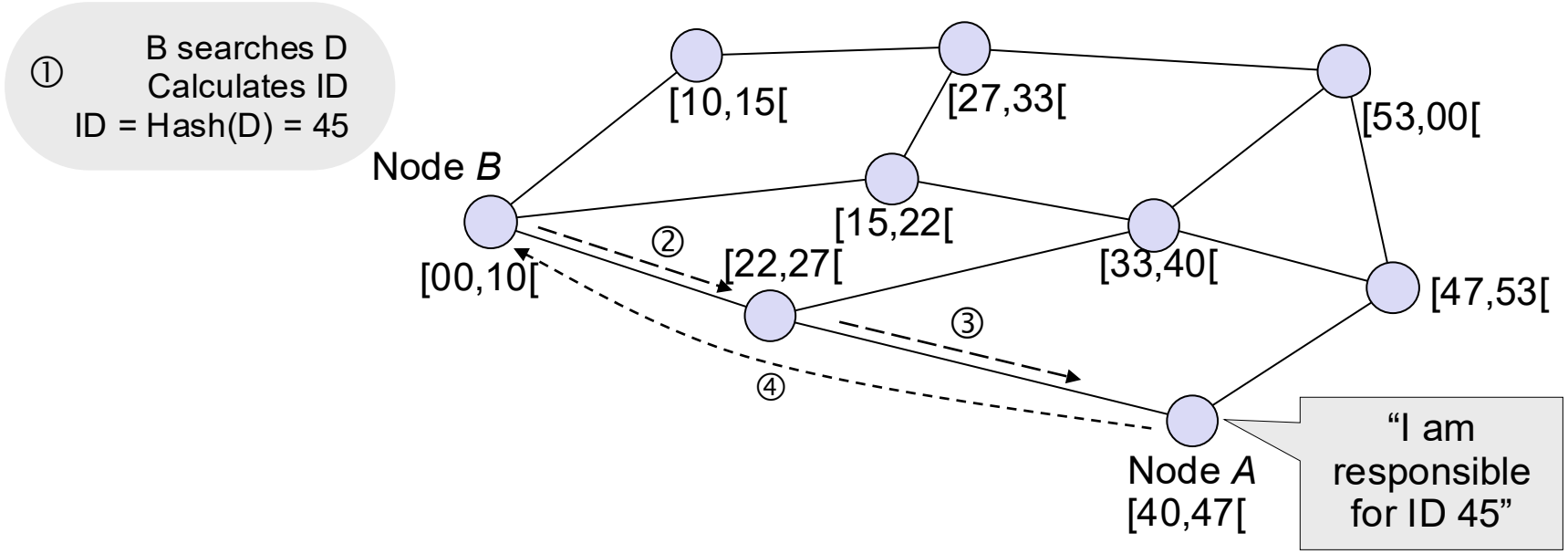
Fully Decentralized Approach

- Construction of an overlay
- Each node responsible for certain range of the ID space
- IDs are calculated using hash functions (e.g. SHA1, MD5)

Routing is done as follows

- ① Node B (requester) calculates ID of D
- ②,③ Node B sends request to neighbor with responsibility interval **closest** to ID
- ④ Node A is responsible for ID 45 and sends requested item D to node B

Strategies for Data Retrieval Approach III: Routing (Distributed Indexing)



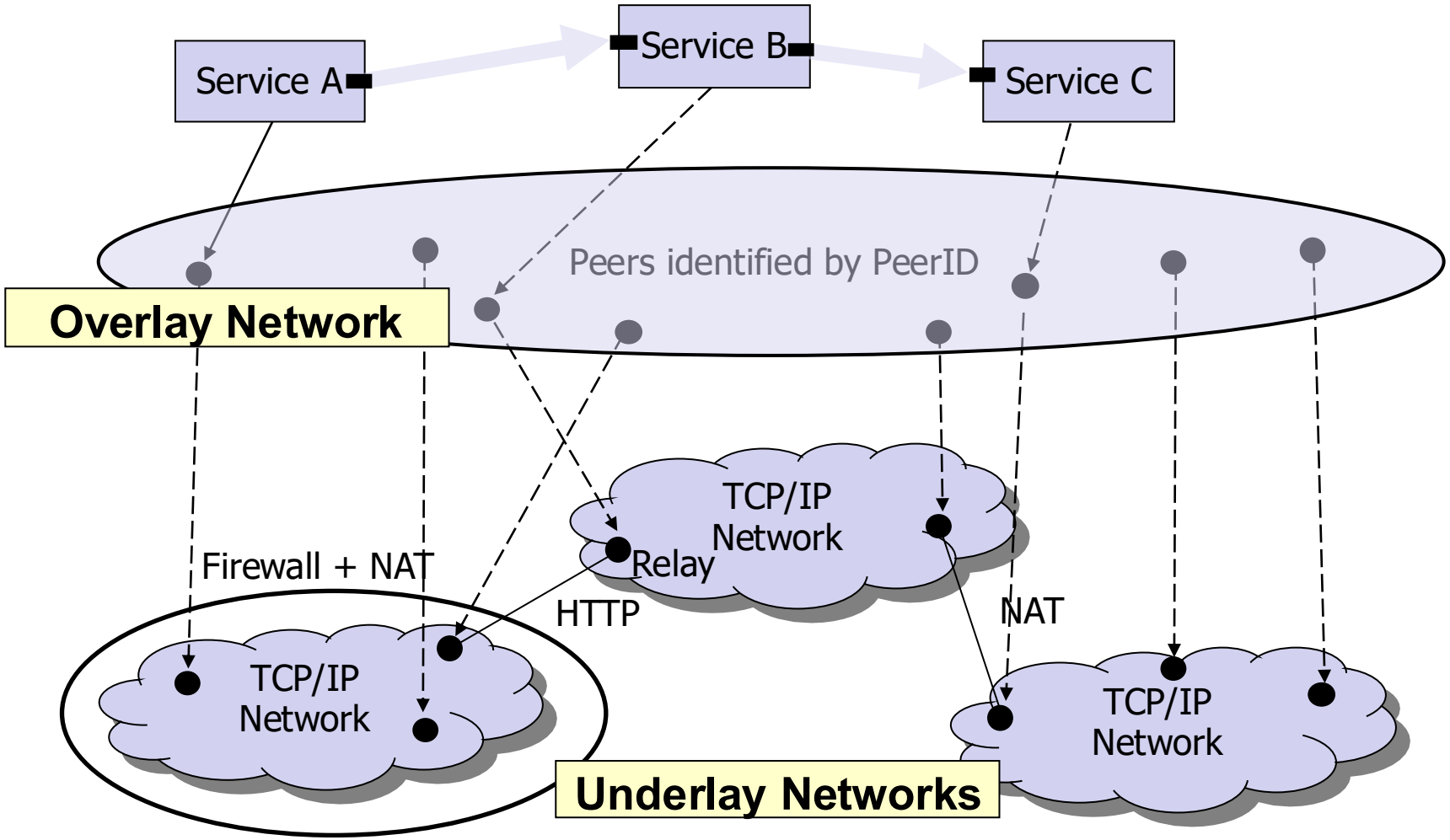
Advantages

- No single point of failure
- Scalable
- Efficient way for retrieving content

Challenges

- Maintenance of ID Space necessary
- Reliable routing
- Load balancing
- Replication of data

Overlay Networks: Layer Model



Picture adapted from Traversat, et.al Project JXTA virtual network

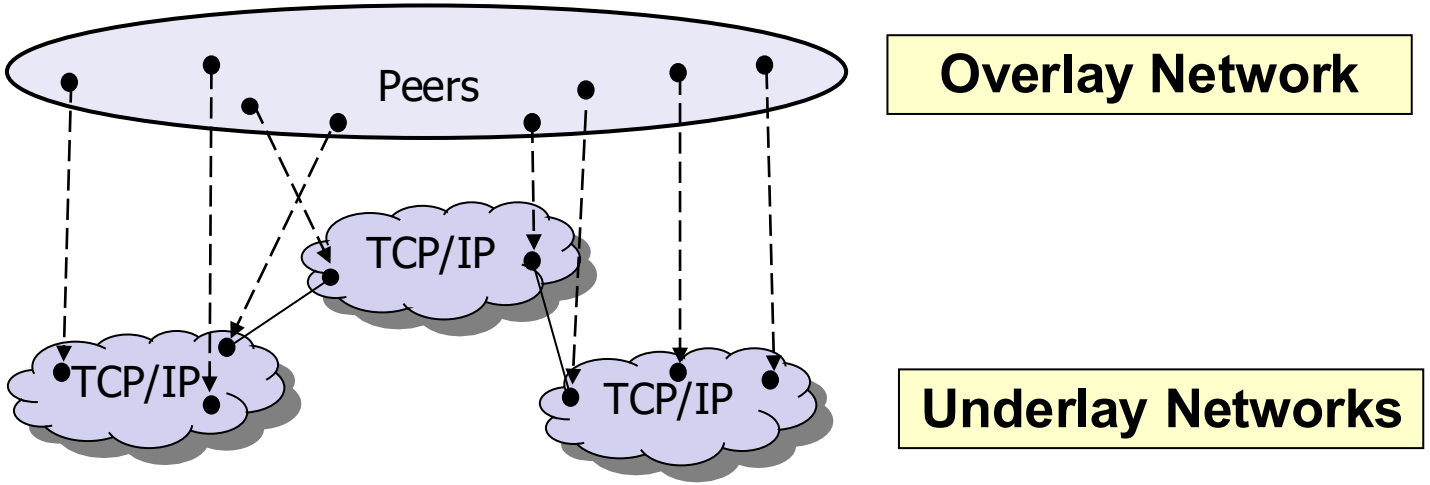
Overlay Networks

A network

- Defines how nodes interact
- Is needed for addressing, routing, …

Overlay network

- Built on top of one or more existing networks
- Adds an additional layer of
  - Abstraction
  - Indirection/virtualization



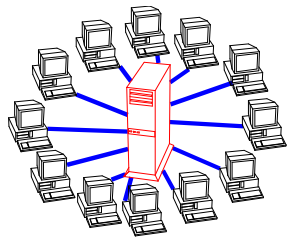
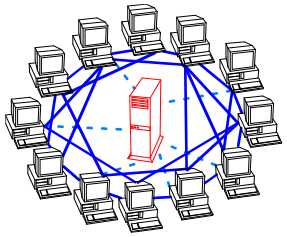
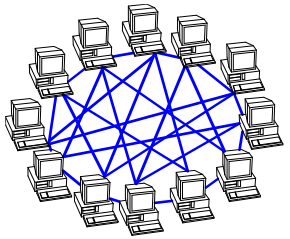
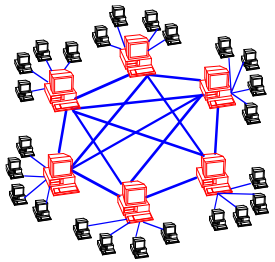
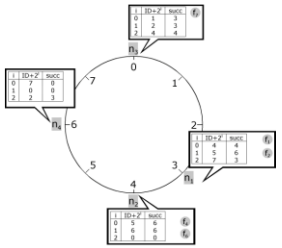
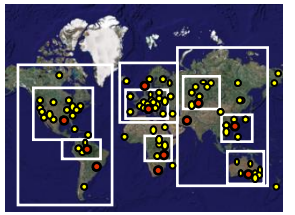
P2P networks form an overlay network

- On top of the Internet (IP network)

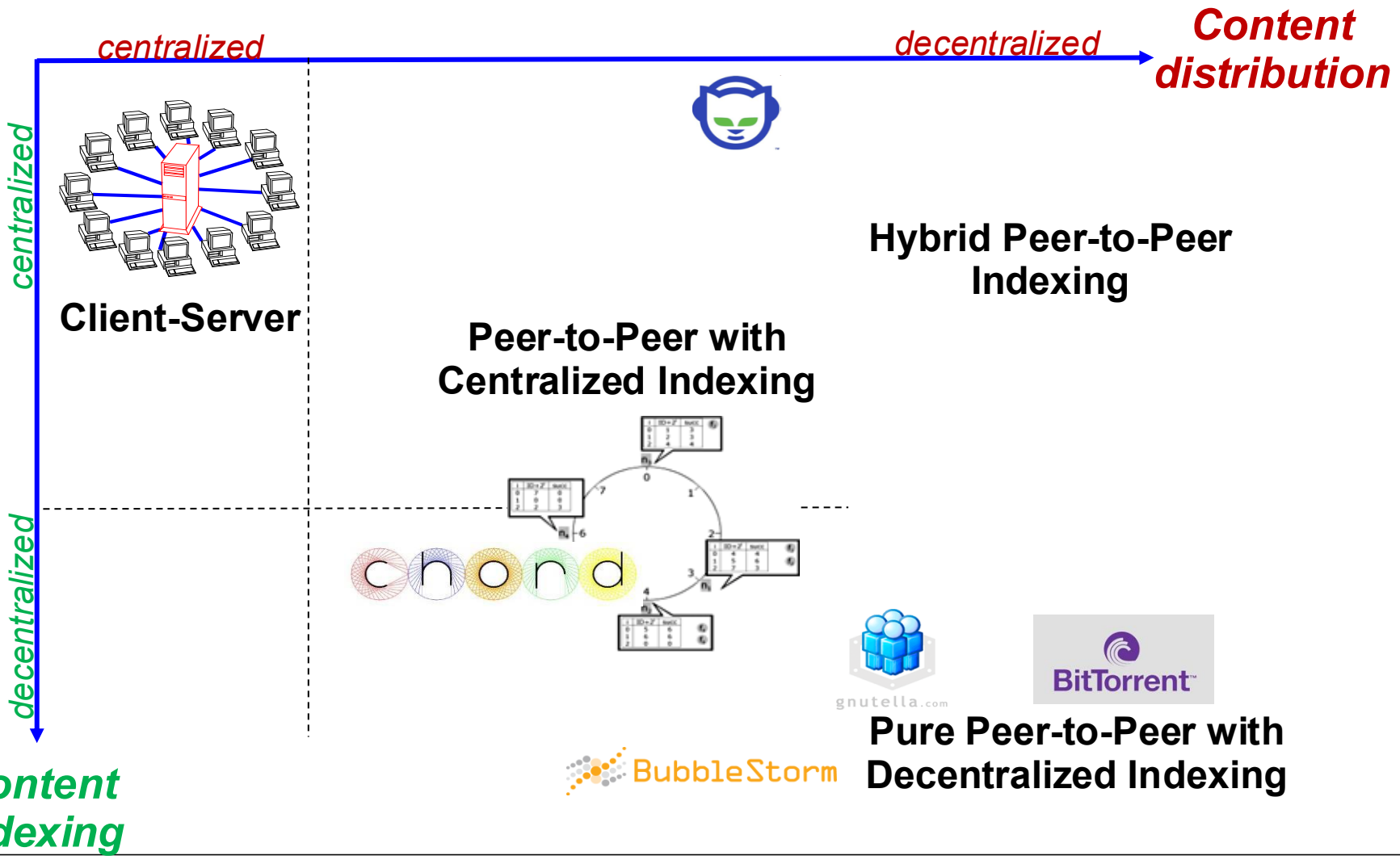
Picture adapted from Traversat, et.al Project JXTA virtual network

Overlay Networks: Structures

TECHNISCHE UNIVERSITÄT DARMSTADT

Client-Server	***Peer-to-Peer***				
1. Server is the central entity and only provider of service and content. → Network managed by the Server 2. Server as the higher performance system. 3. Clients as the lower performance system Example: WWW	1. Resources are shared between the peers 2. Resources can be accessed directly from other peers 3. Peer is provider and requestor (Servent concept)				
	Unstructured P2P			***Structured P2P***	
	Centralized P2P	*Pure P2P*	*Hybrid P2P*	*DHT-Based*	*Hybrid P2P*
	1. All features of Peer-to-Peer included 2. Central entity is necessary to provide the service 3. Central entity is some kind of index/group database Example: Napster	1. All features of Peer-to-Peer included 2. Any terminal entity can be removed without loss of functionality 3. → no central entities Examples: Gnutella 0.4, Freenet	1. All features of Peer-to-Peer included 2. Any terminal entity can be removed without loss of functionality 3. → dynamic central entities Examples: Gnutella 0.6, Fasttrack, edonkey	1. All features of Peer-to-Peer included 2. Any terminal entity can be removed without loss of functionality 3. → No central entities 4. Connections in the overlay are “fixed” 5. Distributed indexing (content is not relocated) Examples: Chord, CAN	1. All features of Peer-to-Peer included 2. Peers are organized in a hierarchical manner 3. Any terminal entity can be removed without loss of functionality Examples: RecNet Globase.KOM
					

Peer-to-Peer Architectures



Structured and Unstructured P2P Networks

TECHNISCHE UNIVERSITÄT DARMSTADT

Unstructured P2P			Structured P2P	
Centralized P2P	Pure P2P	Hybrid P2P	DHT-Based	Hybrid P2P

Unstructured P2P Networks

- Objects have no special identifier
- Location of desired object a **priori not known**
- Each peer is only responsible for objects it submitted

Structured P2P Networks

- Peers and objects have identifiers
- Objects are stored on peers according to their ID:
  responsibleFor(ObjID) = PeerID
- Distributed indexing points to object location

Search:

- Find all (or some) objects in the P2P network which fit the given criteria

vs….

… vs.

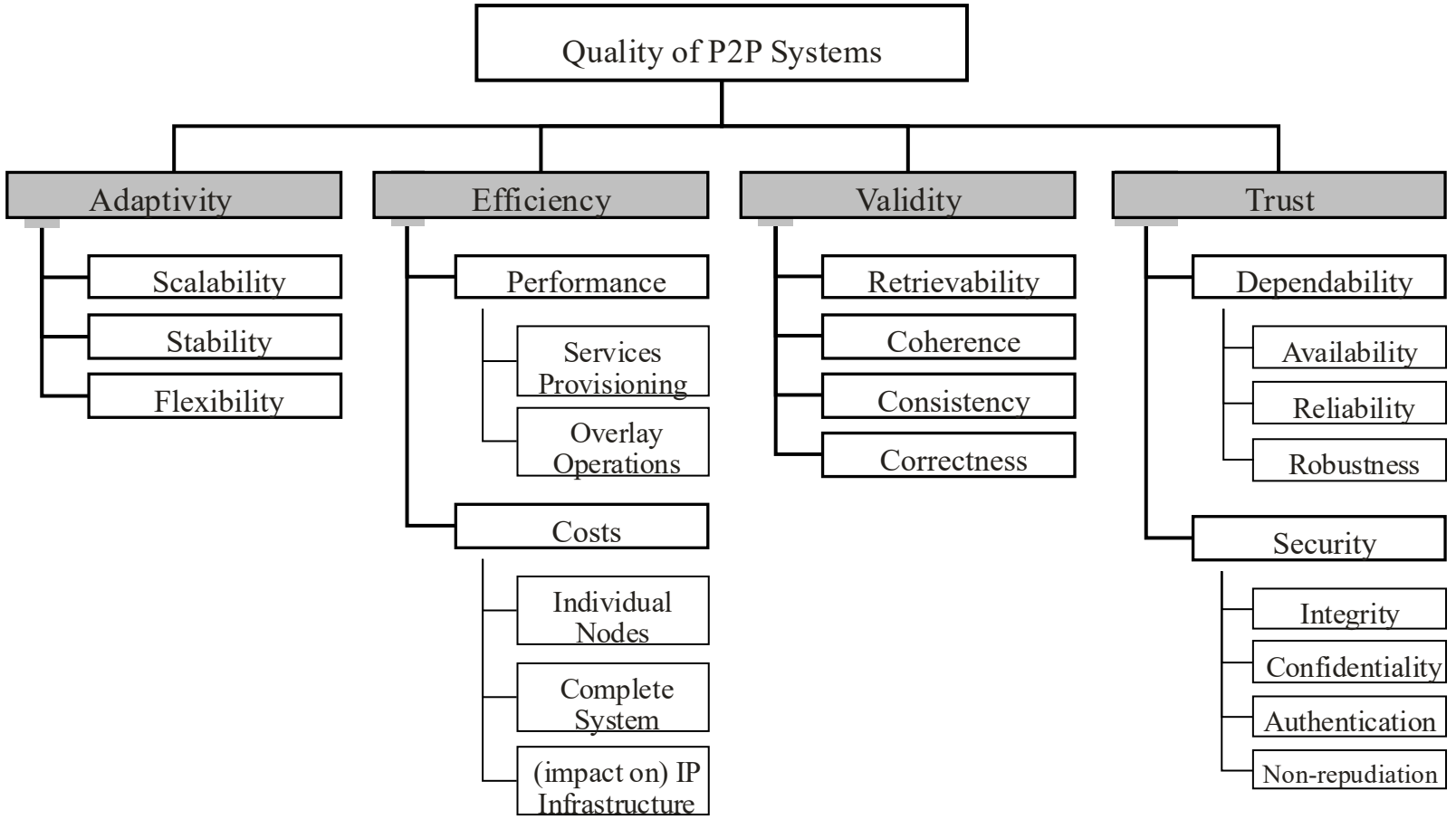
Lookup / Addressing:

- Retrieve the object which is identified with a given identifier

Overlay Networks: Requirements and Design

TECHNISCHE UNIVERSITÄT DARMSTADT

To handle trade-offs among several P2P requirements



Requirements and Metrics for Overlay Networks:

TECHNISCHE UNIVERSITÄT DARMSTADT

Trade-offs of the Requirements

Time – Space
- e.g. local information vs. complete replication of indices

Security – Privacy
- e.g. fully logged operations vs. totally untraceable

Efficiency – Completeness
- e.g. exact key-based matching vs. partial matching (use of wildcards)

Scope – Network load
- With TTL (time to live)
- e.g. TTL based requests vs. exhaustive search

Efficiency – Autonomy
- e.g. hierarchical vs. pure P2P overlays

Reliability – Low maintenance overhead
- e.g. deterministic vs. probabilistic operations

Metrics for Searching and Addressing

Probability of success
- Structured
  - Protocols guarantee results, if target exists
    - Assuming absence of malicious peers
- Unstructured
  - Protocols require exhaustive search

Protocol metrics
- Average number of messages per node
- Visited nodes
- Peak number of messages
- Congestion

Quality of results
- Completeness (are all results returned)
- Correctness (are all returned entries valid)
- Operation latency (time needed to solve the query)

Search Mechanisms in P2P Overlays

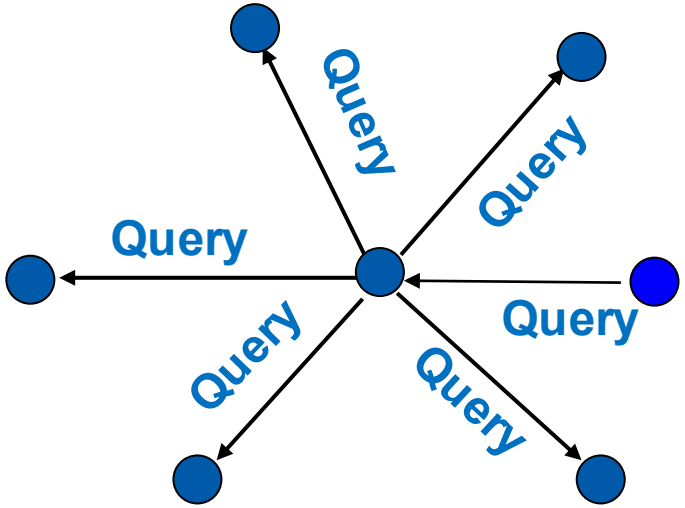
- A. Broadcast
- B. Expanding Ring
- C. Random Walk
- D. Rendezvous Idea

A. Search Mechanisms: Broadcast

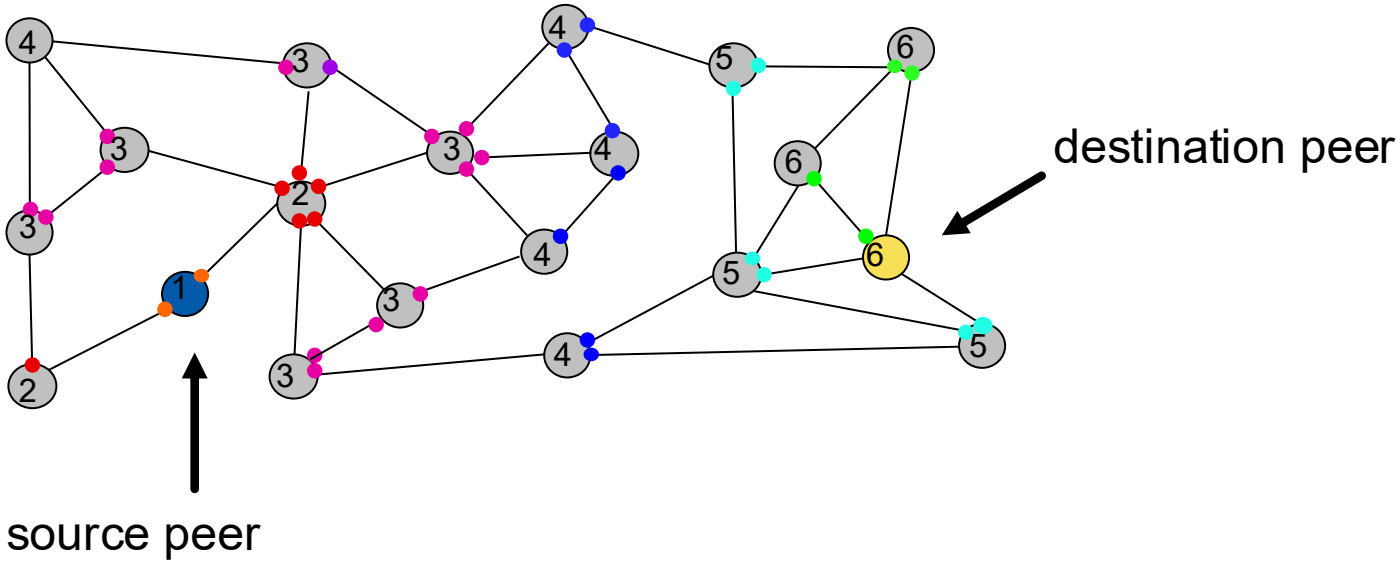
TECHNISCHE UNIVERSITÄT DARMSTADT

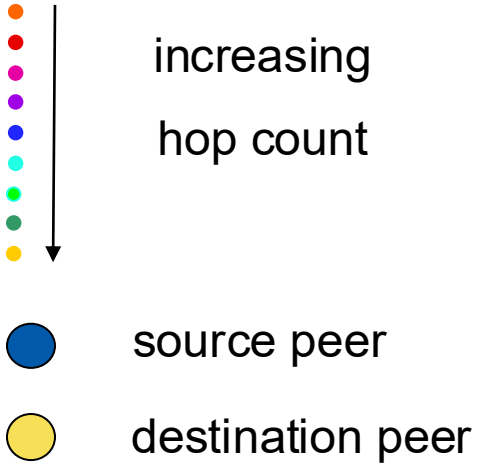
Breadth-first search (BFS)

- Use system-wide maximum TTL to control communication overhead
- Send a message to all neighbors
  - except the one who delivered the incoming message
- Store message identifiers of routed messages
  - or use non-oblivious messages to avoid retransmission cycles



Example for Broadcast





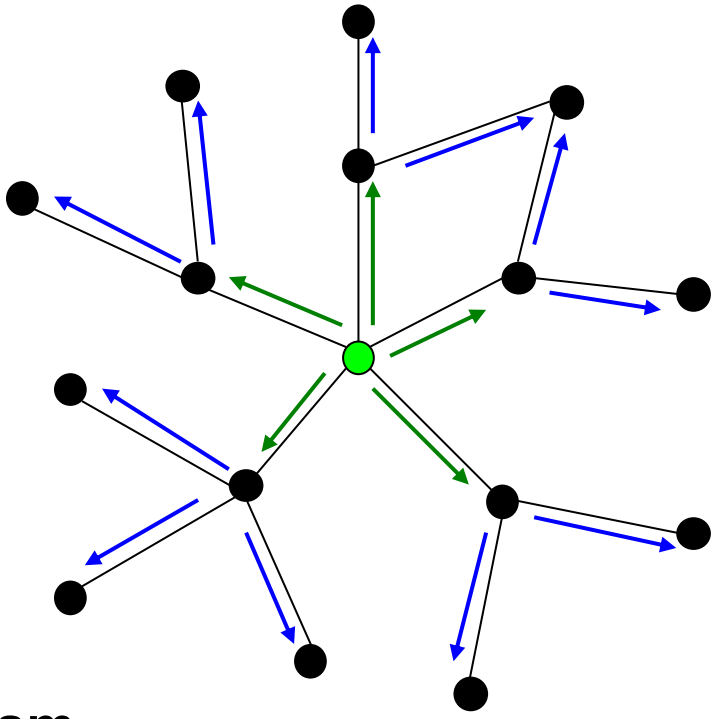
Overhead

- Large, here 43 messages sent

Length of the path:

- 5 hops

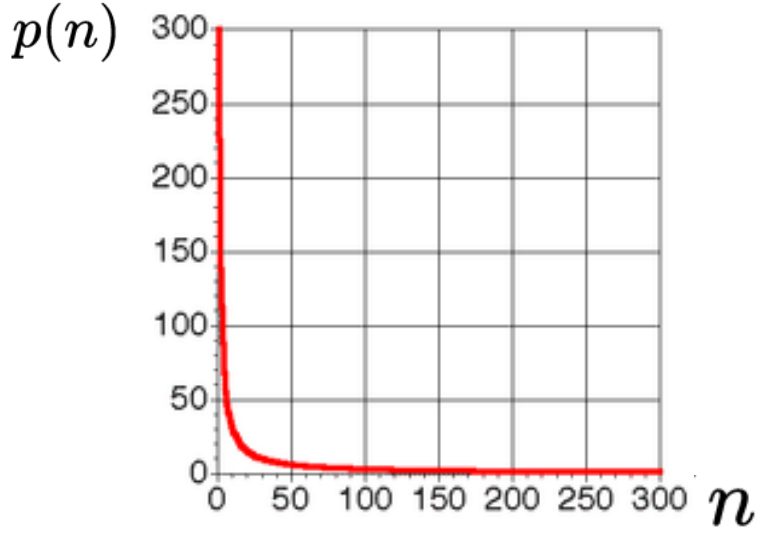
B. Search Mechanisms: Expanding Ring



Mechanism

- Successive floods with increasing TTL
  - Start with small TTL
  - If no success increase TTL
  - .. etc.

Zipf-law example $p(n) \sim \frac{1}{n}$



Properties

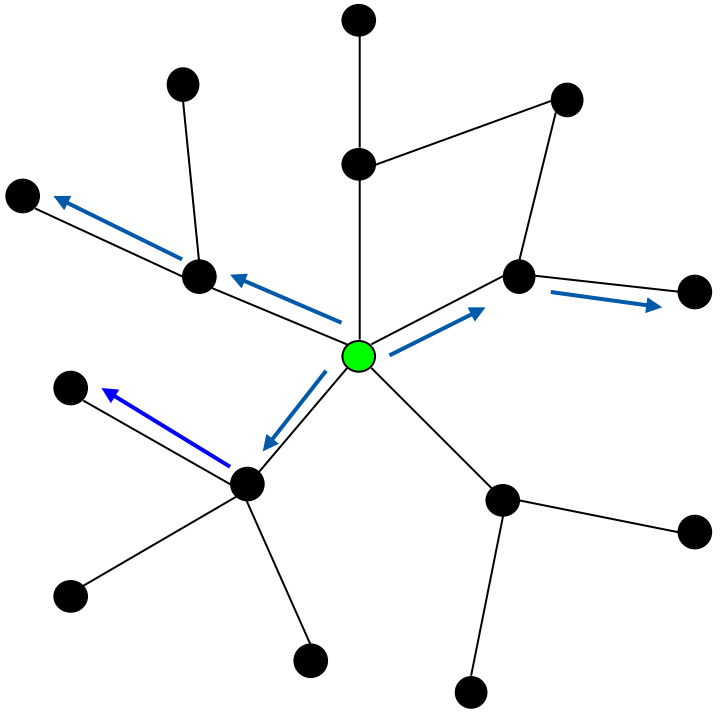
- Improved performance
  - If objects, follow Zipf law popularity distribution and are located accordingly
- Message overhead is high

C. Search Mechanisms: Random Walk

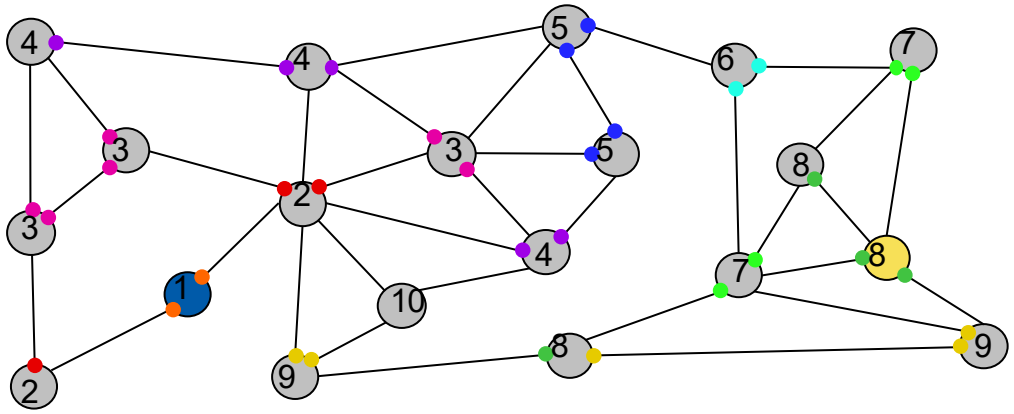
TECHNISCHE UNIVERSITÄT DARMSTADT

Random walks

- Forward the query to a randomly selected neighbor
  - Message overhead is reduced significantly
  - Increased latency
- Multiple random walks
  (k-query messages)
  - Reduces latency
  - Generates more load
- Termination mechanism
  - TTL-based
  - Periodically checking requester before next submission



Example Random Walk





increasing

hop count

source peer

destination peer

Random walk with n=2

- (each incoming message is sent twice out)

Overhead

- Smaller, here e.g., 30 messages sent until destination is reached

Length of the path found

- e.g.
  - 7 hops

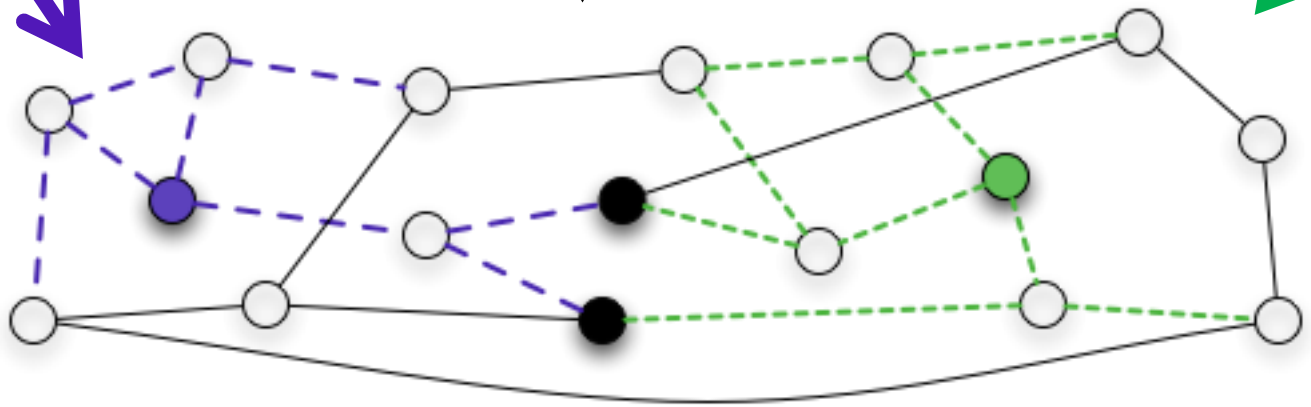
D. Search Mechanisms: Rendezvous Point

TECHNISCHE UNIVERSITÄT DARMSTADT

Requesting node (blue/dark grey on left side) propagates his query to all neighbors within a predefined range

A query hit can be found at the Rendezvous Point (black)

Storing node (green/light grey on right side) propagates content to all nodes within a predefined range



Source:http://www.dvs.tu-darmstadt.de/research/bubblestorm/

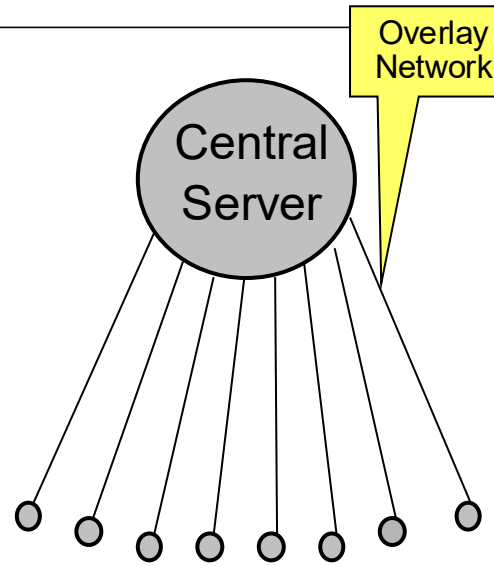
Unstructured Centralized P2P Networks

TECHNISCHE UNIVERSITÄT DARMSTADT

Client-Server	***Peer-to-Peer***				
1. Server is the central entity and only provider of service and content. → Network managed by the Server 2. Server as the higher performance system. 3. Clients as the lower performance system Example: WWW	1. Resources are shared between the peers 2. Resources can be accessed directly from other peers 3. Peer is provider and requestor (Servent concept)				
		Unstructured P2P		***Structured P2P***	
	Centralized P2P	*Pure P2P*	*Hybrid P2P*	*DHT-Based*	*Hybrid P2P*
	1. All features of Peer-to-Peer included 2. Central entity is necessary to provide the service 3. Central entity is some kind of index/group database Example: Napster	1. All features of Peer-to-Peer included 2. Any terminal entity can be removed without loss of functionality 3. → no central entities Examples: Gnutella 0.4, Freenet	1. All features of Peer-to-Peer included 2. Any terminal entity can be removed without loss of functionality 3. → dynamic central entities Examples: Gnutella 0.6, Fasttrack, edonkey	1. All features of Peer-to-Peer included 2. Any terminal entity can be removed without loss of functionality 3. → No central entities 4. Connections in the overlay are “fixed” 5. Distributed indexing (content is not relocated) Examples: Chord, CAN	1. All features of Peer-to-Peer included 2. Peers are organized in a hierarchical manner 3. Any terminal entity can be removed without loss of functionality Examples: RecNet Globase.KOM

Compare to R.Schollmeier and J.Eberspächer, TU München

Centralized P2P Networks



Central index server, maintaining index:

- What:
  - Object name, file name, criteria (ID3) …
- Where:
  - (IP address, Port)
- Search engine, combining both information
- Global view of the network

Normal peer, maintaining the objects:

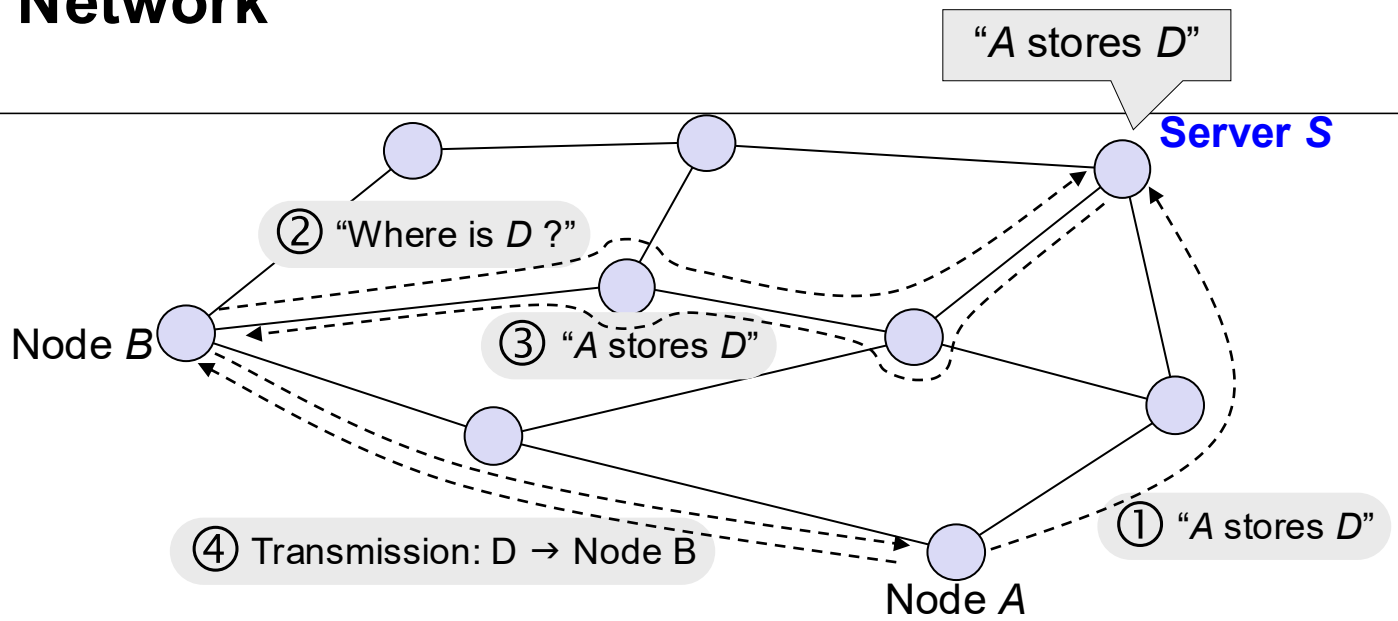
- Each peer maintains only its own objects
- Decentralized storage (content at the edges)
- File transfer between clients (decentralized)

Issues:

- Unbalanced costs: central server is the bottleneck
- Security: server is single point of attack

Centralized P2P Network

Similar to Data Retrieval Approach I



Simple strategy:

- Central server stores information about locations
  - ① Node A (provider) tells server that it stores item D
  - ② Node B (requester) asks server S for location of D
  - ③ Server S tells B that node A stores item D
  - ④ Node B requests item D from node A

Unstructured Centralized P2P Networks

TECHNISCHE UNIVERSITÄT DARMSTADT

Client-Server	***Peer-to-Peer***				
1. Server is the central entity and only provider of service and content. → Network managed by the Server 2. Server as the higher performance system. 3. Clients as the lower performance system Example: WWW	1. Resources are shared between the peers 2. Resources can be accessed directly from other peers 3. Peer is provider and requestor (Servent concept)				
		Unstructured P2P		***Structured P2P***	
	Centralized P2P	*Pure P2P*	*Hybrid P2P*	*DHT-Based*	*Hybrid P2P*
	1. All features of Peer-to-Peer included 2. Central entity is necessary to provide the service 3. Central entity is some kind of index/group database Example: Napster	1. All features of Peer-to-Peer included 2. Any terminal entity can be removed without loss of functionality 3. → no central entities Examples: Gnutella 0.4, Freenet	1. All features of Peer-to-Peer included 2. Any terminal entity can be removed without loss of functionality 3. → dynamic central entities Examples: Gnutella 0.6, Fasttrack, edonkey	1. All features of Peer-to-Peer included 2. Any terminal entity can be removed without loss of functionality 3. → No central entities 4. Connections in the overlay are “fixed” 5. Distributed indexing (content is not relocated) Examples: Chord, CAN	1. All features of Peer-to-Peer included 2. Peers are organized in a hierarchical manner 3. Any terminal entity can be removed without loss of functionality Examples: RecNet Globase.KOM

Compare to R.Schollmeier and J.Eberspächer, TU München

Distributed / Pure P2P Systems

Characteristics

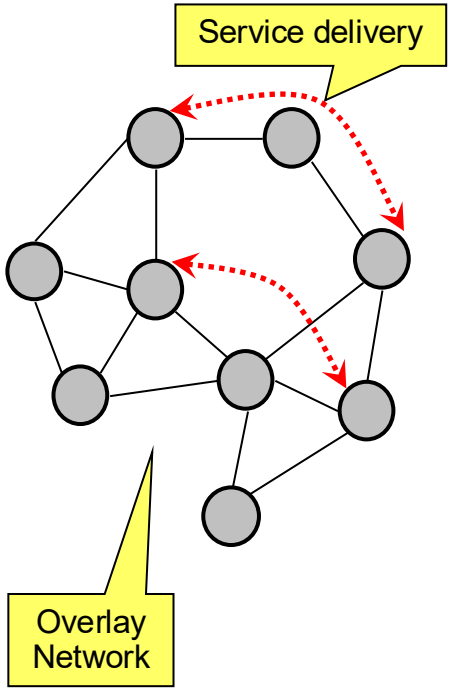
- All peers are equal
  - (in their roles)
- Search mechanism is provided by cooperation of all peers
- Local view of the network

Organic growing:
Just append to current network

- No special infrastructure element needed

Motivation:

- To provide robustness
- To have self organization



Tasks to solve

1. To connect to the network

- No central index server
  → Joining strategies needed
- To join
  → knowledge of at least 1 peer in the network
- Local view of network
  → advertisements needed

2. To search

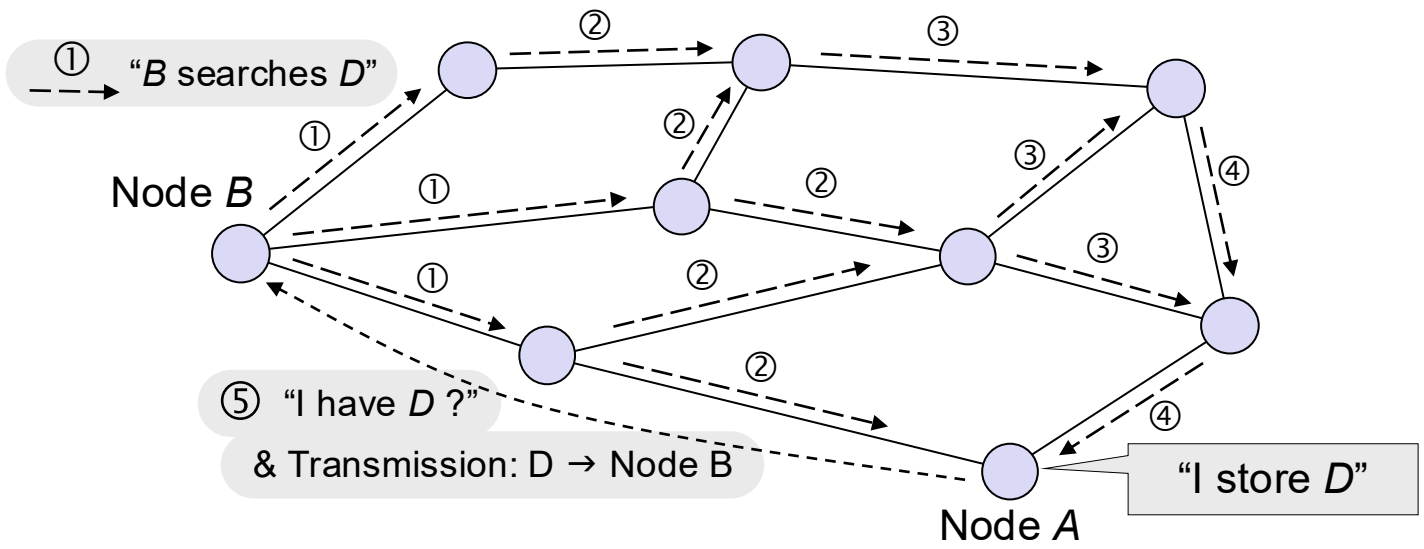
- Different search strategies available
- Providing different benefits & drawbacks

3. To deliver the service

- Establish connection to other node(s)
- Peer-to-peer communication

Search in Distributed / Pure P2P Systems

Similar to Data Retrieval Approach II

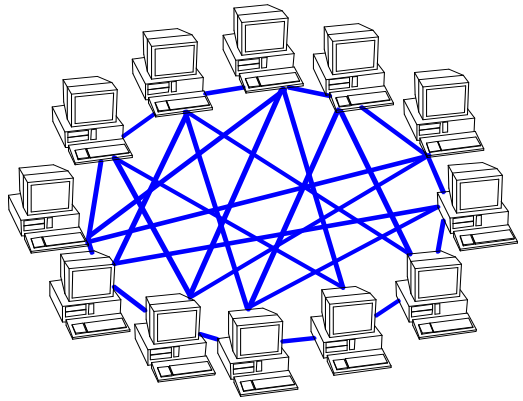


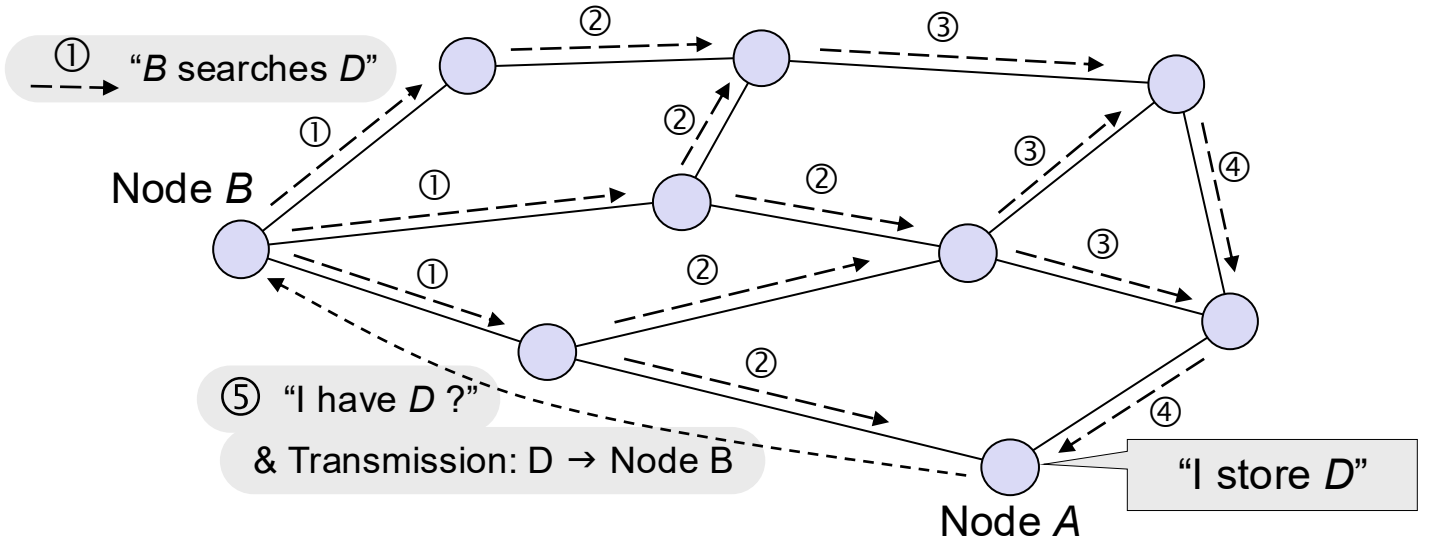
Fully Decentralized Approach

- No information about location of data at intermediate systems
- Necessity for broad search
  - ① Node B (requester) asks neighboring nodes for item D
  - ②-④ Nodes forward request to further nodes (breadth-first search / flooding)
  - ⑤ Node A (provider of item D) sends D to requesting node B

The content of this slide has been adapted from "Peer-to-Peer Systems and Applications", edt. By Steinmetz, Wehrle

Properties of Distributed / Pure P2P Networks





Benefits:

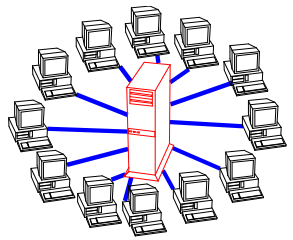
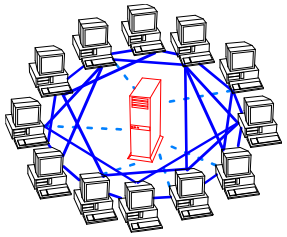
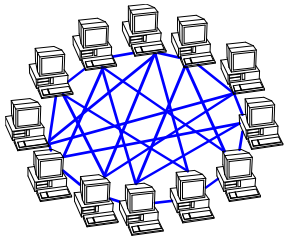
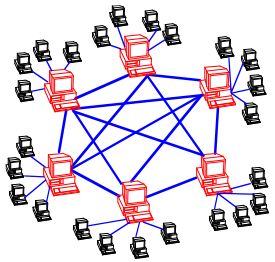
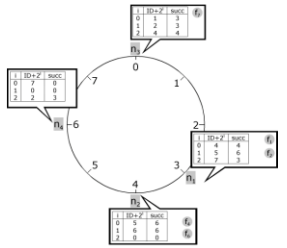
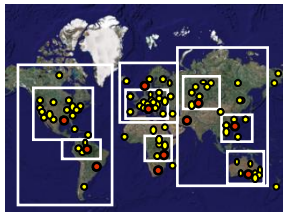
- Robustness: Every peer is dispensable
  - Switch off peer → no effect for network
- Balanced costs:
  - Each peer (generally) contributes the same
- Self organization

Drawbacks:

- Slow and expensive search
  - Flooding (to all connected nodes) is used to distribute information
- Finding all objects is not guaranteed
  - Object out of reach for search query

Overlay Networks: Structures

TECHNISCHE UNIVERSITÄT DARMSTADT

Client-Server	***Peer-to-Peer***				
1. Server is the central entity and only provider of service and content. → Network managed by the Server 2. Server as the higher performance system. 3. Clients as the lower performance system Example: WWW	1. Resources are shared between the peers 2. Resources can be accessed directly from other peers 3. Peer is provider and requestor (Servent concept)				
	Unstructured P2P			***Structured P2P***	
	Centralized P2P	*Pure P2P*	*Hybrid P2P*	*DHT-Based*	*Hybrid P2P*
	1. All features of Peer-to-Peer included 2. Central entity is necessary to provide the service 3. Central entity is some kind of index/group database Example: Napster	1. All features of Peer-to-Peer included 2. Any terminal entity can be removed without loss of functionality 3. → no central entities Examples: Gnutella 0.4, Freenet	1. All features of Peer-to-Peer included 2. Any terminal entity can be removed without loss of functionality 3. → dynamic central entities Examples: Gnutella 0.6, Fasttrack, edonkey	1. All features of Peer-to-Peer included 2. Any terminal entity can be removed without loss of functionality 3. → No central entities 4. Connections in the overlay are "fixed" 5. Distributed indexing (content is not relocated) Examples: Chord, CAN	1. All features of Peer-to-Peer included 2. Peers are organized in a hierarchical manner 3. Any terminal entity can be removed without loss of functionality Examples: RecNet Globase.KOM
					

Compare to R.Schollmeier and J.Eberspächer, TU München

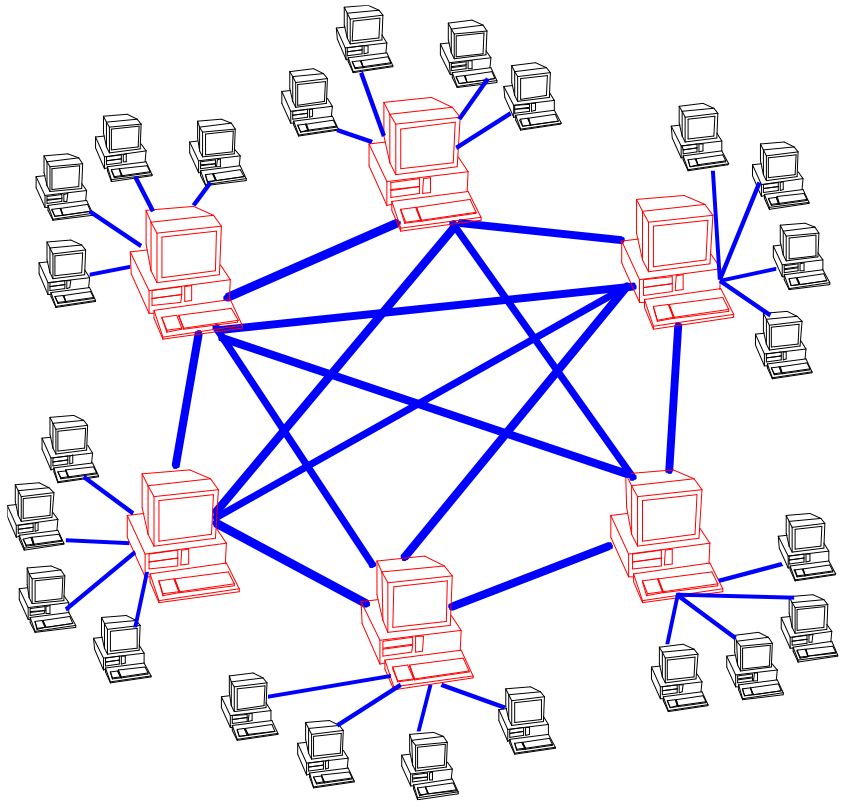
Principles

Approach:
To combine best of both worlds

- Robustness by distributed indexing
- Fast searches by server queries

Components

- Supernodes
  - Mini servers / super peers
  - Used as servers for queries
    - To build a sub-network between supernodes
    - Queries distributed at sub-network between supernodes
- “Normal” peers
  - Have only overlay connections to supernodes



Advantages

- More robust than centralized solutions
- Faster searches than in pure P2P systems

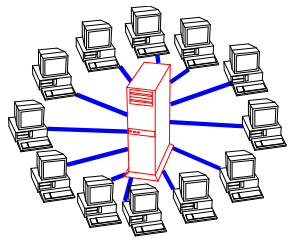
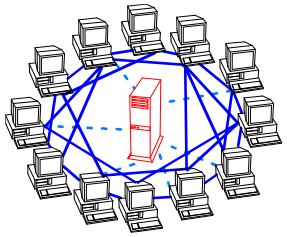
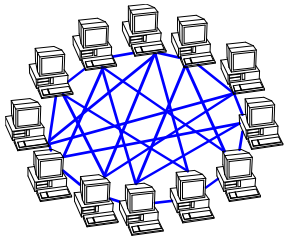
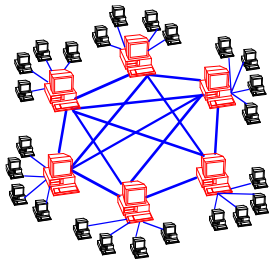
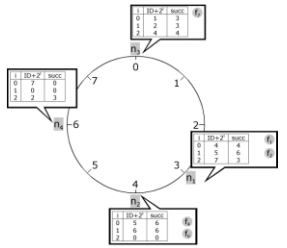
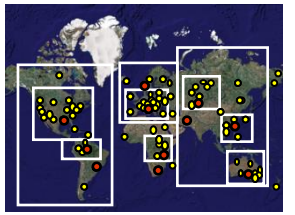
Disadvantages

- Need of algorithms to choose reliable supernodes

Picture from R.Schollmeier and J.Eberspächer, TU München

Overlay Networks: Structures

TECHNISCHE UNIVERSITÄT DARMSTADT

Client-Server	***Peer-to-Peer***				
1. Server is the central entity and only provider of service and content. → Network managed by the Server 2. Server as the higher performance system. 3. Clients as the lower performance system Example: WWW	1. Resources are shared between the peers 2. Resources can be accessed directly from other peers 3. Peer is provider and requestor (Servent concept)				
	Unstructured P2P			***Structured P2P***	
	Centralized P2P	*Pure P2P*	*Hybrid P2P*	*DHT-Based*	*Hybrid P2P*
	1. All features of Peer-to-Peer included 2. Central entity is necessary to provide the service 3. Central entity is some kind of index/group database Example: Napster	1. All features of Peer-to-Peer included 2. Any terminal entity can be removed without loss of functionality 3. → no central entities Examples: Gnutella 0.4, Freenet	1. All features of Peer-to-Peer included 2. Any terminal entity can be removed without loss of functionality 3. → dynamic central entities Examples: Gnutella 0.6, Fasttrack, edonkey	1. All features of Peer-to-Peer included 2. Any terminal entity can be removed without loss of functionality 3. → No central entities 4. Connections in the overlay are “fixed” 5. Distributed indexing (content is not relocated) Examples: Chord, CAN	1. All features of Peer-to-Peer included 2. Peers are organized in a hierarchical manner 3. Any terminal entity can be removed without loss of functionality Examples: RecNet Globase.KOM
					

Compare to R.Schollmeier and J.Eberspächer, TU München

Structured P2P Networks

How to actually find content in such a network?

Number of content providers is (potentially) very large

- Just storing the location of every provider is infeasible
- Would also lead to significant management traffic due to frequent join/leaves of nodes

Idea: Every peer has a high resolution view on its proximity, while having low resolution in the distance

- Requests are routed closer to the desired destination, as nodes close by have a better view

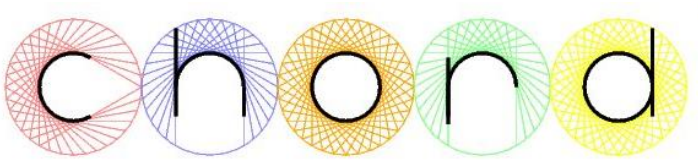


Structured Overlay Networks - Chord

TECHNISCHE UNIVERSITÄT DARMSTADT

Chord

- Efficient Lookup Network
- protocol and algorithm for a peer-to-peer distributed hash table
  - A distributed hash table
    - stores key-value pairs by assigning keys to different computers (known as "nodes");
    - a node will store the values for all the keys for which it is responsible.
- Chord specifies how
  - keys are assigned to nodes
  - a node can discover the value for a given key by first locating the node responsible for that key.



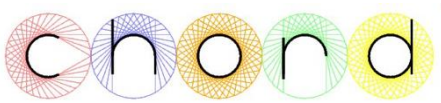
Usage

- In 2021 e.g., Identity resolution system design based on Dual-Chord algorithm for industrial Internet of Things

History

- one of the four original distributed hash table protocols,
  - along with CAN, Tapestry, and Pastry
- introduced in 2001, see
  - https://pdos.csail.mit.edu/papers/ton:chord/paper-ton.pdf
  - Ion Stoica, Robert Morris, David Karger, Frans Kaashoek, and Hari Balakrishnan
- was developed mainly at MIT

Chord





Chord uses SHA-1 hash function

- Results in a 160-bit object/node identifier
- Same hash function for objects and nodes

e.g. Ip addr.	SHA-1 hash value
130.83.58.11	27856a552c522d22e56ec26195154ae06bf04cb3
130.83.58.12	27856a552c522d22e56ec26195154ae06bf04cb3
2001:41b8:83f:4711::1000:1/64	ff0649bd44adfded79f4d3b48ce93c23fcf51665

Node ID hashed from e.g., IP address

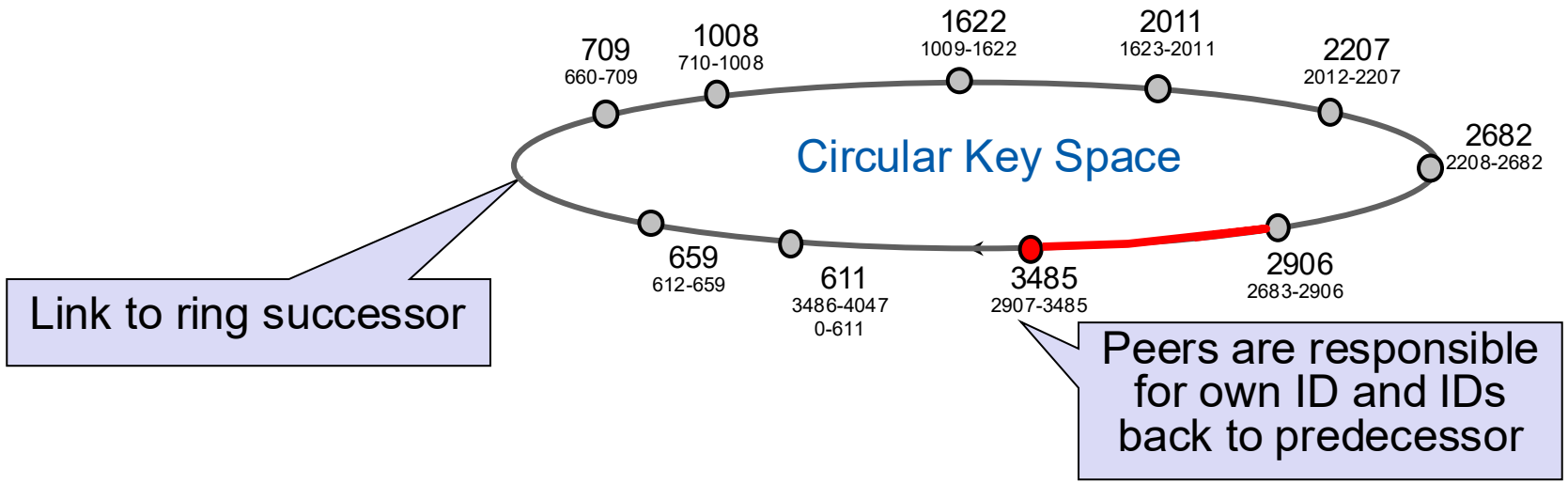
Object ID hashed from object name

- Object names assumed to be known

Chord is organized in a ring which wraps around

- Nodes keep track of predecessor and successor
  - System invariant for valid network operation
- Node responsible for
  - Objects between its predecessor and itself
- Fingers used to enable efficient content addressing
  - O(log(N)) fingers lead to lookup operation of O(log(N)) length

Chord: Network Topology



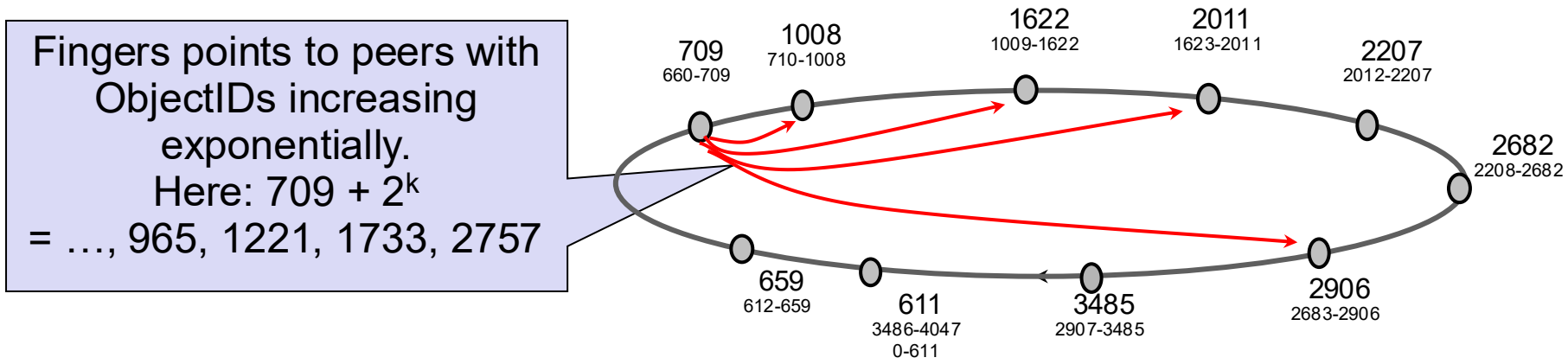
Uses SHA-1 (secure hash algorithm) to map

- IP address/object name onto
- 160 Bit ID

Basic ring topology

- Successor/ Predecessor

Chord: Network Topology



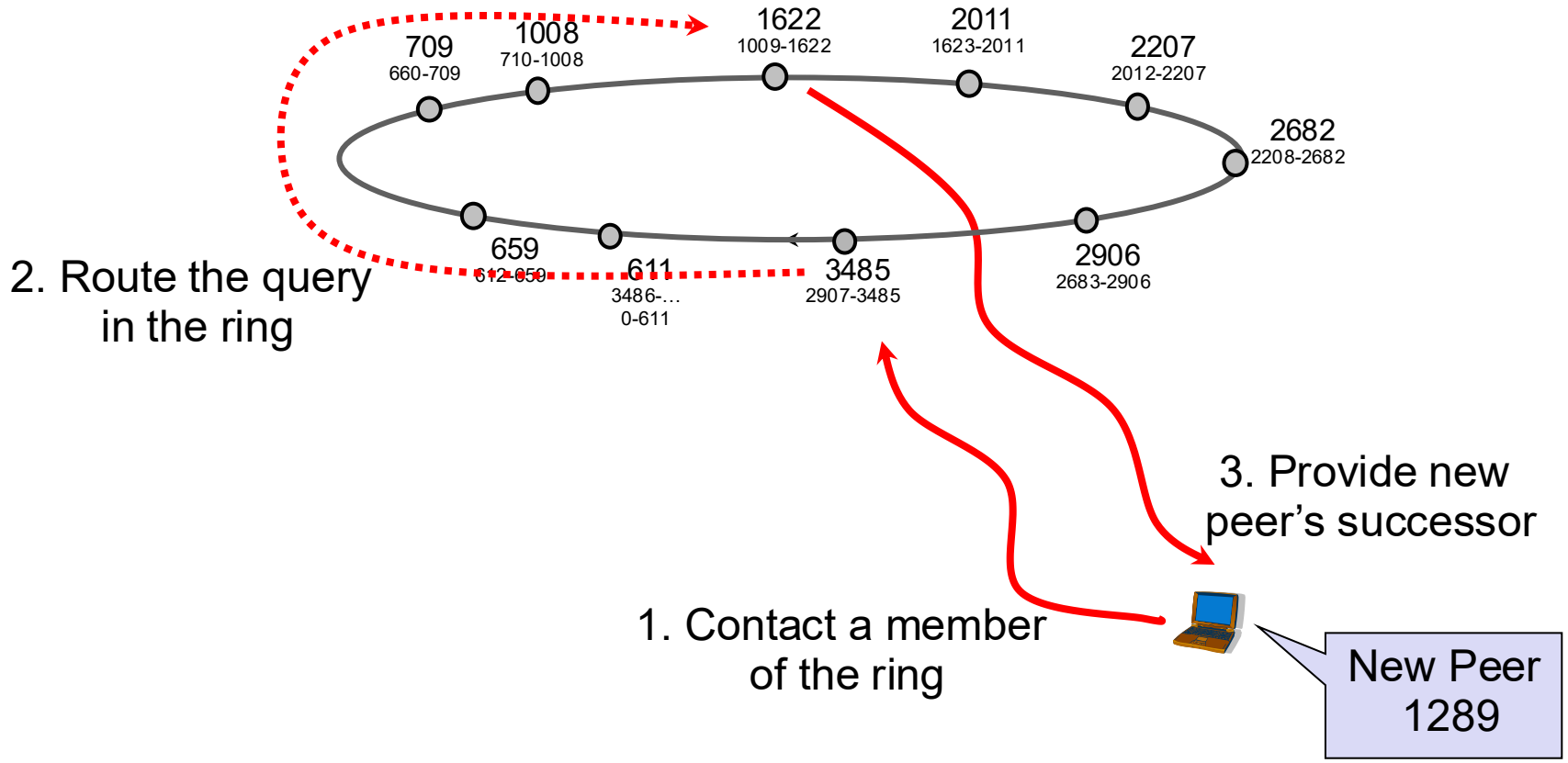
Enhanced topology

- k^{th} finger of Peer n is shortcut pointing to peers being responsible for Object ID $(n + 2^k)$
- k ranges from 0 to log(N)
- O(log(N)) fingers lead to lookup operation of O(log(N))

Chord: Join Procedure (1)

TECHNISCHE UNIVERSITÄT DARMSTADT

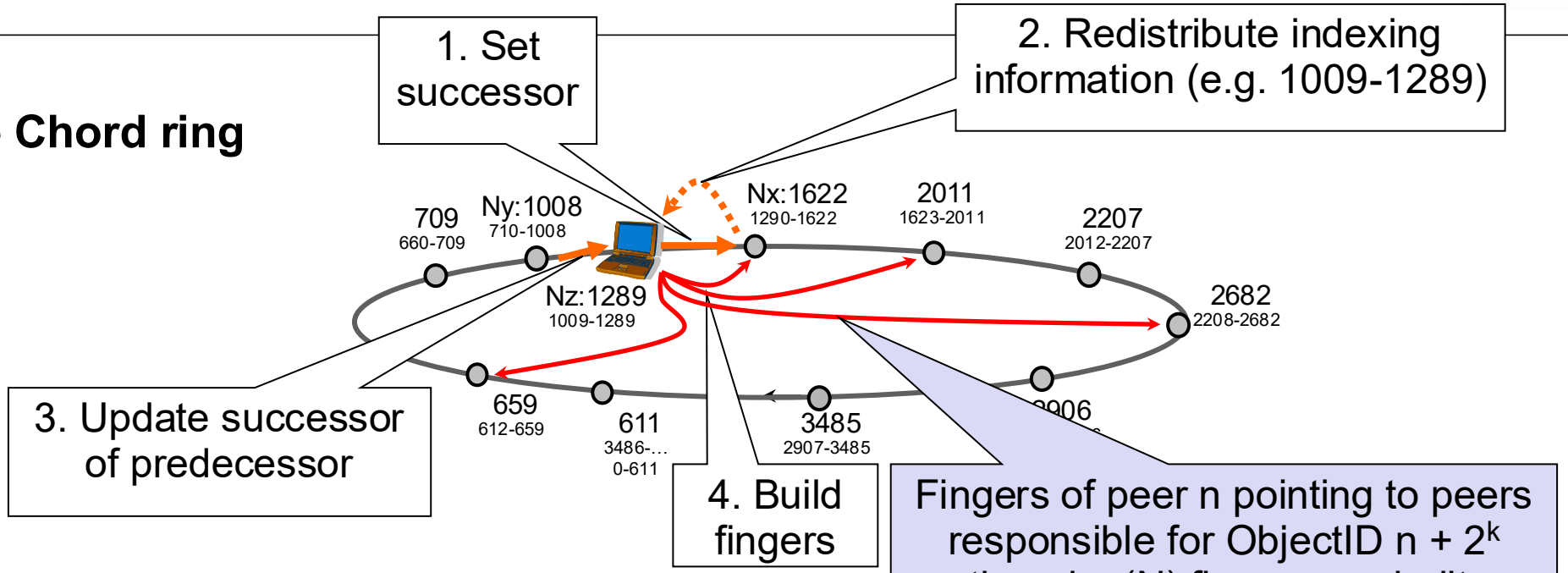
Request to join the Chord ring



Chord: Join Procedure (2)

TECHNISCHE UNIVERSITÄT DARMSTADT

Request to join the Chord ring



1. & 2. Notify Successor

Actions:
N_Z: Set Successor (N_X)
N_Z: Notify N_X
N_X: Set Predecessor
N_X: Copy items (index) to N_Z

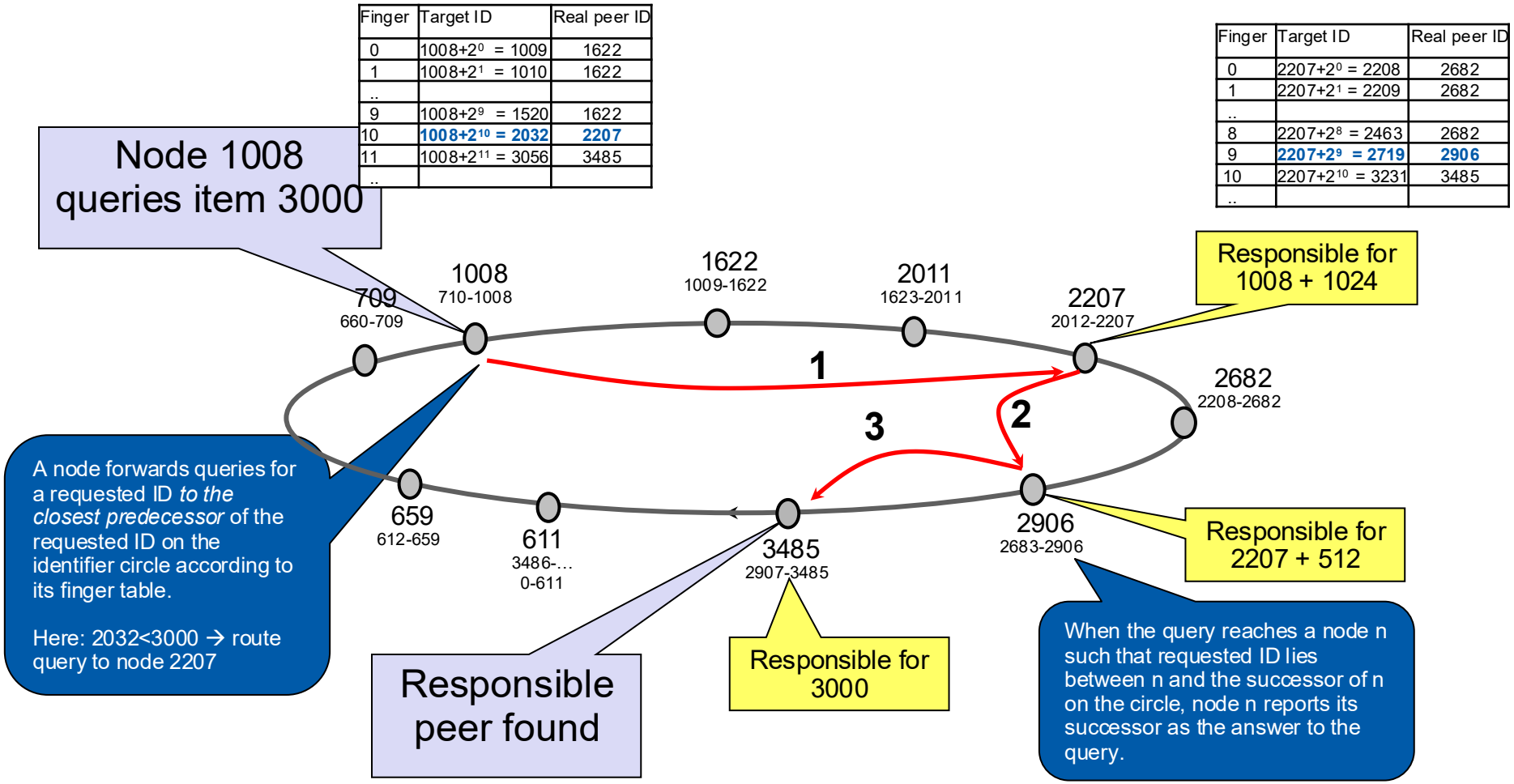
3. & 4. Stabilize

Actions:
N_Y: Ask Predecessor of N_X
N_Y: Set Successor (N_Z)
N_Y: Notify N_Z
N_Z: Set Predecessor (N_Y)
N_X: Clear moved items
All: Fix Fingers

Chord: Addressing Content

Query

- Contains the hash value of the queried content
- On each step the distance from the destination is halved (remember fingers)



Properties of Chord

TECHNISCHE UNIVERSITÄT DARMSTADT

Advantages

Efficient look up functionality
- Messages are routed within O(log N) steps

Low maintenance overhead

Easy to implement

Intuitive concept due to ring structure

Disadvantages

Not churn resistant
- Chord ring is likely to fail
- Insufficient stabilization mechanism

No support for heterogeneity
- All peers are treated equally
- Overloading of peers may happen

No built-in security mechanisms
- Sensitive to malicious nodes

P2P Networks

Kademlia

TECHNISCHE UNIVERSITÄT DARMSTADT

Peer-to-Peer protocols do not rely on a single coordination instance

- No single-point-of-failure
- Consider the failure of nodes in their design
- One example: Kademlia

History

- distributed hash table for decentralized peer-to-peer computer networks
- designed by Petar Maymounkov and David Mazières in 2002.

Kademlia

- specifies the structure of the network and
- the exchange of information through node lookups.
- Kademlia nodes communicate among themselves using UDP.
- A virtual or overlay network is formed by the participant nodes.

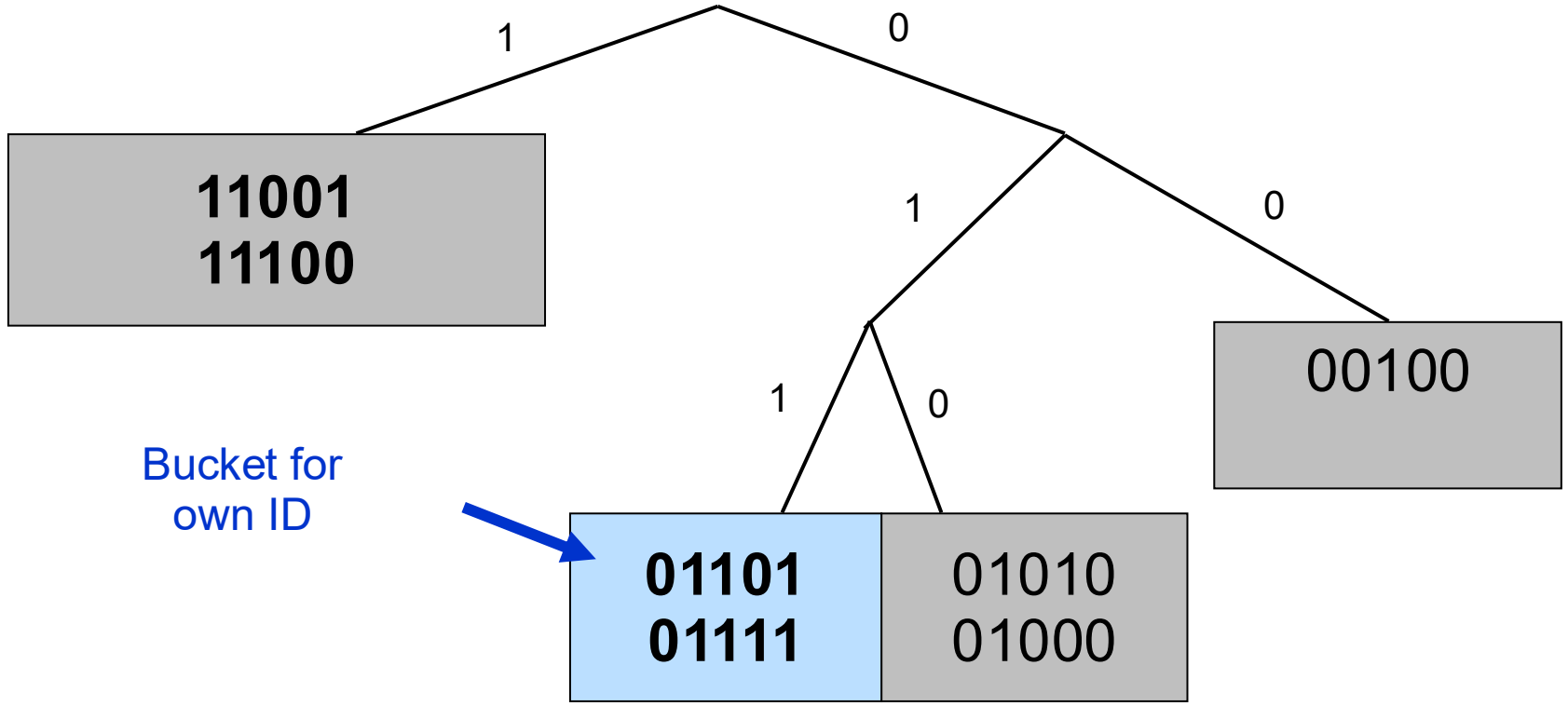
node ID

- Each node is identified by a number or node ID
- algorithm uses the node ID to locate values
  - usually file hashes or keywords.
- provides a direct map to file hashes and that node stores information on where to obtain the file or resource.
- Distance between nodes is calculated using XOR

Use lookup messages to maintain the overlay network

- To exchange routing table entries with look up messages
- i.e. to learn useful routing information from received lookup requests

Kademlia Routing Table



Basic idea: Have many information about close neighbors, only few about distant neighbors

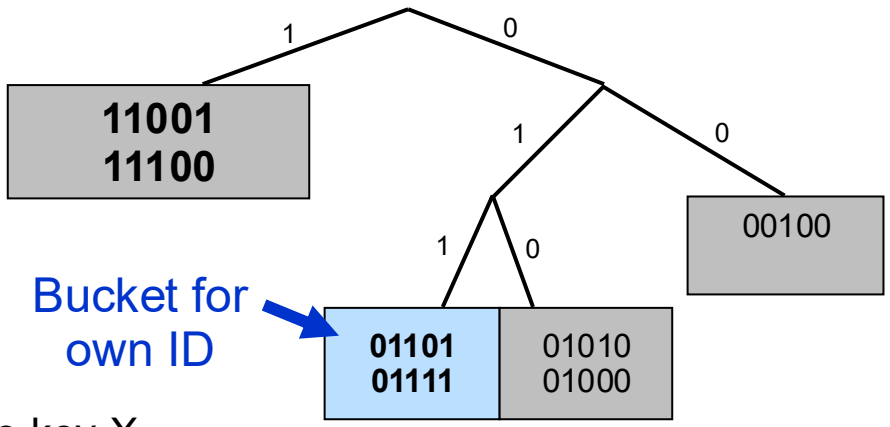
Concept of the Kademlia Routing Process

TECHNISCHE UNIVERSITÄT DARMSTADT

Structure of routing table

- Every node maintains a binary tree like routing table
- Tree branches along the local node ID
- Every leaf is a bucket with k entries

Example routing table for node 01110 with bucket factor k = 2



Lookup procedure for a key X

Step 1: Traverse routing table tree and pick bucket with node IDs closest to key X

Step 2: Put node IDs in a node list

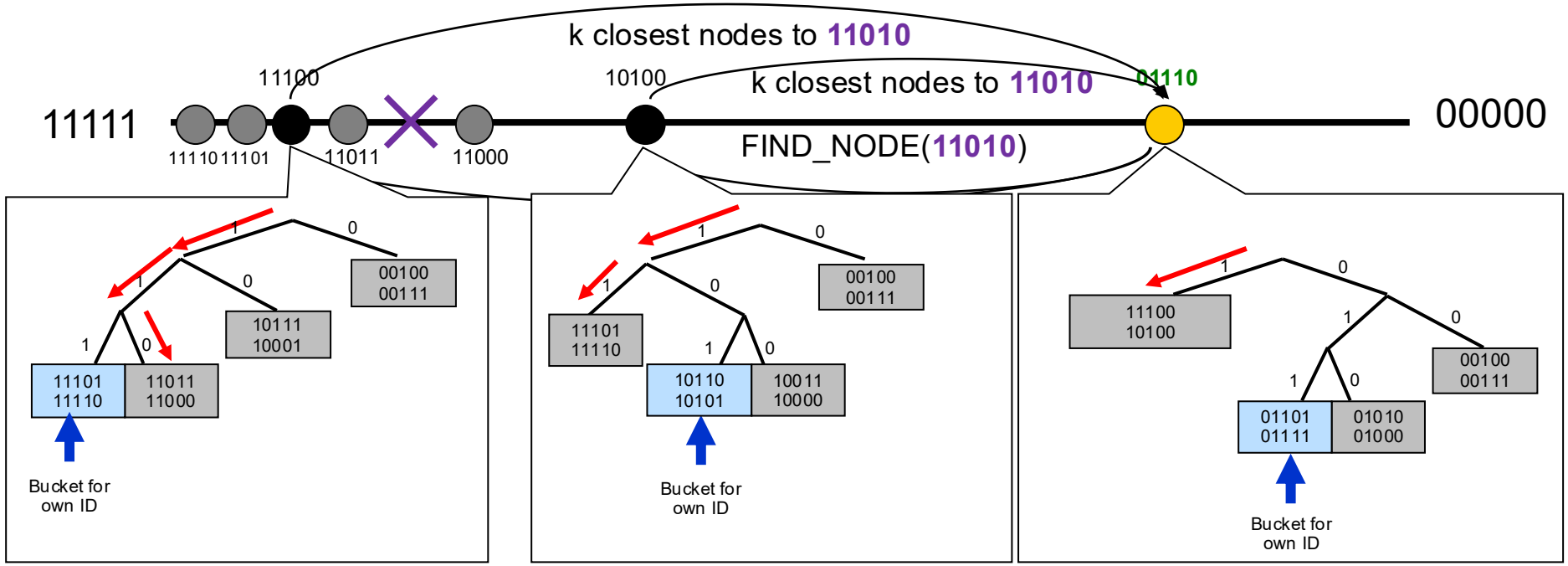
Step 3: Send parallel requests to the first α unvisited nodes in the node list

Step 4: Put received node IDs in node list

Step 5: Repeat Step 3 and 4 until set of k-closest nodes do not change anymore

Step 6: Pick k-closest nodes from node list and send store or get data request to them

Routing from 01110 to 11010, α = 2 (parallel look-ups), k = 2 (bucket factor)

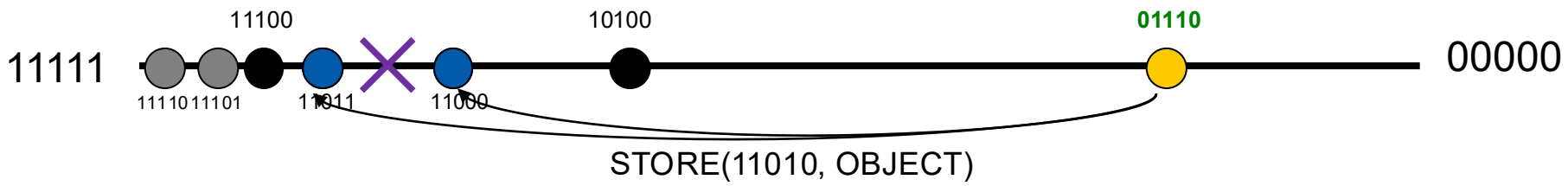


Newly learned closest nodes

Already known nodes

Routing from 01110 to 11010, α = 2 (parallel look-ups), k = 2 (bucket factor)

Send additional requests, stop once no new nodes discovered → store/access object



- FIND_NODE(KEY):
  - Recipient returns <IP Address, UDP Port, Node ID> triples for k closest nodes he knows about
- FIND_VALUE(KEY):
  - Like FIND_NODE, except if recipient already stores the value, then the value is returned instead of k closest nodes
- PING(IP ADDRESS)
  - Probes a node to see if it is online
- STORE(KEY, VALUE)
  - Instructs a node to store a <key,value> pair

Newly learned closest nodes

Already known nodes

k closest nodes

Kademlia Routing Tree

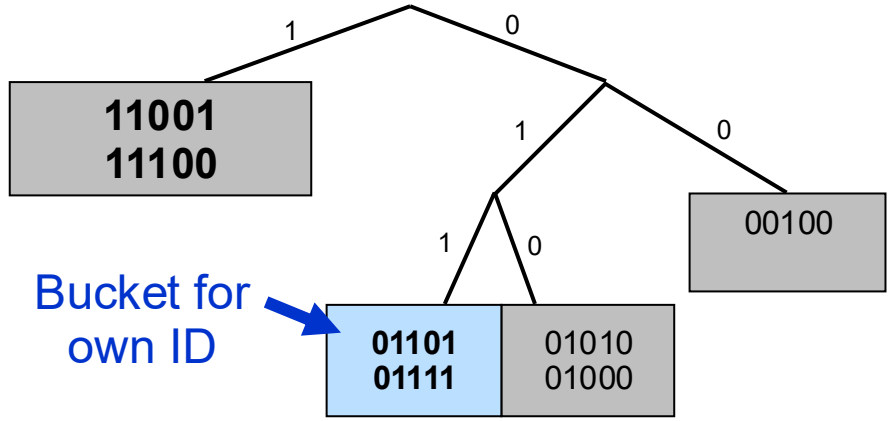
TECHNISCHE UNIVERSITÄT DARMSTADT

The routing tree connects the Kademlia ids to real IP addresses

Kademlia nodes constantly listen for incoming messages

- If a new id is received, it may be added to the routing tree

Example routing table for node 01110 with bucket factor k = 2



Kademlia Routing Tree

The routing tree connects the Kademlia ids to real IP addresses

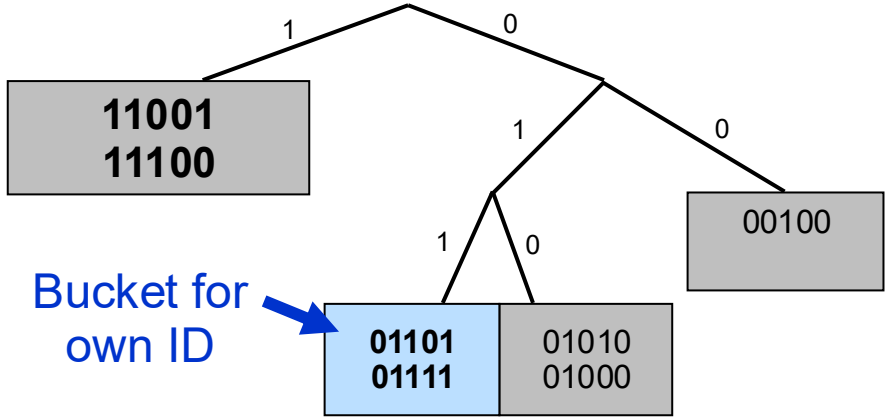
Kademlia nodes constantly listen for incoming messages

- If a new id is received, it may be added to the routing tree

A new id is added, if

- The bucket id in which the id belongs is not full (< k)

Example routing table for node 01110 with bucket factor k = 2



Message from 11.12.13.14 with ID 00110

Kademlia Routing Tree

TECHNISCHE UNIVERSITÄT DARMSTADT

The routing tree connects the Kademlia ids to real IP addresses

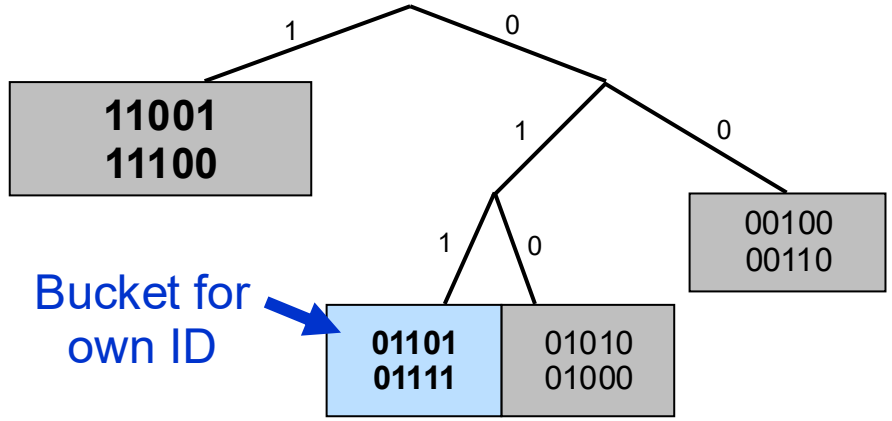
Kademlia nodes constantly listen for incoming messages

- If a new id is received, it may be added to the routing tree

A new id is added, if

- The bucket id in which the id belongs is not full (< k)
- The bucket id in which the id belongs is full, but also the bucket in which the own id resides
  - The bucket is then split into two buckets
  - The new id is added

Example routing table for node 01110 with bucket factor k = 2



Message from 12.13.14.15 with ID 01100

Kademlia Routing Tree

TECHNISCHE UNIVERSITÄT DARMSTADT

The routing tree connects the Kademlia ids to real IP addresses

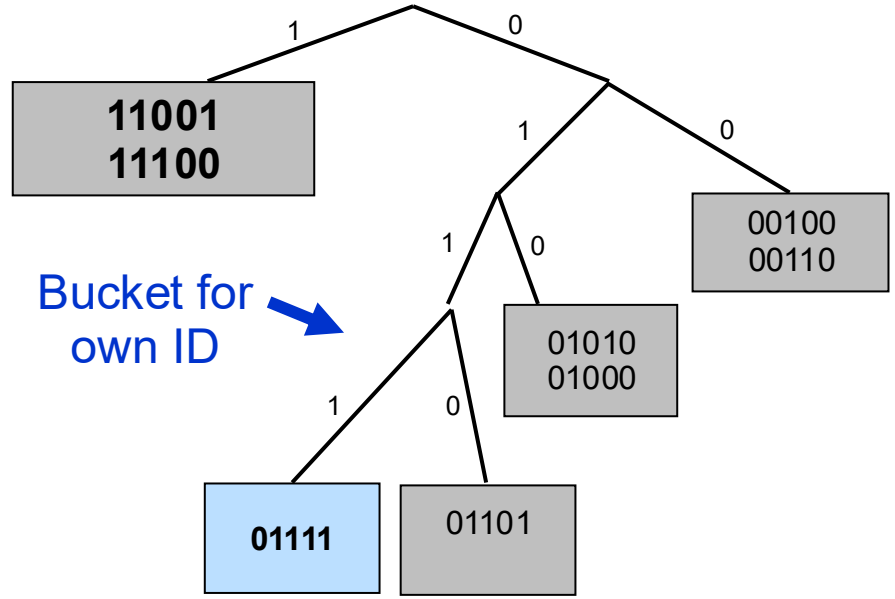
Kademlia nodes constantly listen for incoming messages

- If a new id is received, it may be added to the routing tree

A new id is added, if

- The bucket id in which the id belongs is not full (< k)
- The bucket id in which the id belongs is full, but also the bucket in which the own id resides
  - The bucket is then split into two buckets
  - The new id is added

Example routing table for node 01110 with bucket factor k = 2



Message from 12.13.14.15 with ID 01100

Kademlia Routing Tree

TECHNISCHE UNIVERSITÄT DARMSTADT

The routing tree connects the Kademlia ids to real IP addresses

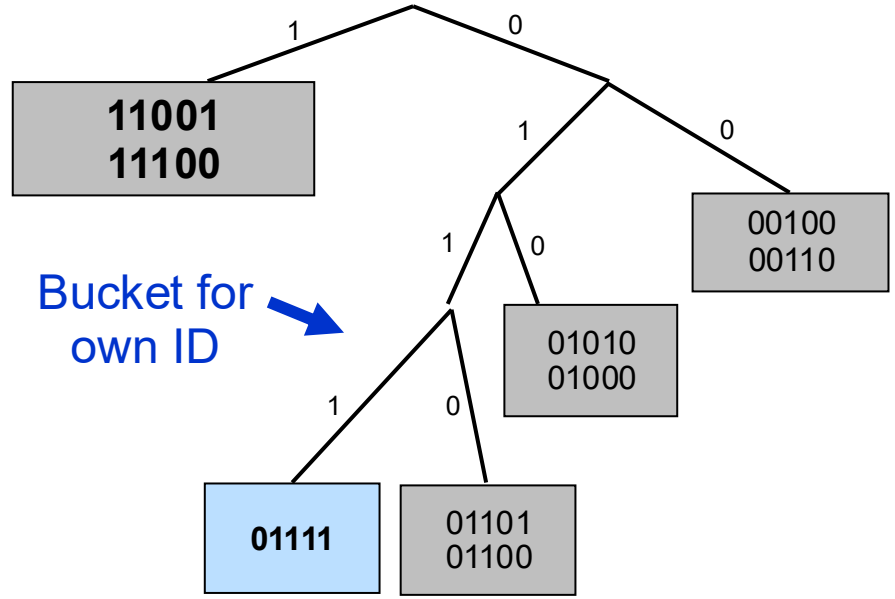
Kademlia nodes constantly listen for incoming messages

- If a new id is received, it may be added to the routing tree

A new id is added, if

- The bucket id in which the id belongs is not full (< k)
- The bucket id in which the id belongs is full, but also the bucket in which the own id resides
  - The bucket is then split into two buckets
  - The new id is added

Example routing table for node 01110 with bucket factor k = 2



Message from 12.13.14.15 with ID 01100

Properties of Kademlia

TECHNISCHE UNIVERSITÄT DARMSTADT

Similar efficiency as Chord-based networks

- Simple distance checks using the XOR metric
- More resilient to disruptions due to redundancy

Actively used even today

- BitTorrent
- IPFS
- …